

Báo cáo Đồ án Tốt nghiệp

Nhóm thực hiện: Omnichannel Inventory and Sales Management System (OISM)

Ngày 18 tháng 9 năm 2026

Mục lục

1	Giới thiệu tổng quan dự án	1
1.1	Tổng quan dự án	1
1.1.1	Thông tin dự án	1
1.1.2	Thành viên nhóm	1
1.2	Bối cảnh và nền tảng đề tài	2
1.3	Hệ thống hiện tại	2
1.3.1	Phần mềm quản lý bán hàng đơn kênh truyền thống	2
1.3.2	Giải pháp ERP/Middleware kết nối đa kênh của bên thứ ba	2
1.4	Cơ hội kinh doanh và vận hành	3
1.5	Tầm nhìn sản phẩm	3
1.6	Phạm vi và giới hạn của dự án	3
1.6.1	Các tính năng chính (theo thiết kế đầy đủ của hệ thống)	3
1.6.2	Phạm vi bàn giao thực tế của nhóm trong học kỳ này	4
1.6.3	Hạn chế và loại trừ	4
2	Kế hoạch Quản lý Dự án	5
2.1	Tổng quan	5
2.1.1	Phạm vi và Ước lượng	5
2.1.2	Mục tiêu dự án	5
2.1.3	Rủi ro dự án	5
2.2	Phương pháp Quản lý	7
2.2.1	Quy trình dự án	7
2.2.2	Quản lý chất lượng	7
2.2.3	Kế hoạch onboard công nghệ	8
2.3	Sản phẩm Bàn giao	8
2.4	Phân công Trách nhiệm	9
2.5	Giao tiếp Dự án	10
2.6	Quản lý Cấu hình	11
2.6.1	Quản lý tài liệu	11
2.6.2	Quản lý mã nguồn & tài liệu thiết kế	11
2.6.3	Công cụ và hạ tầng	11
3	Software Requirement Specification	13
3.1	Phạm vi của hệ thống	13
3.2	Product Overview	13
3.2.1	Mô tả luồng dữ liệu và ràng buộc nghiệp vụ	14
3.3	User Requirements	14
3.3.1	Actors	14
3.3.2	Use Cases	15
3.3.3	System Functional Overview	16

3.3.4	FR-AUTH: Xác thực & Đa khách thuê (Multi-tenant Authentication)	19
3.3.5	FR-PROD: Quản lý sản phẩm & biến thể (Product & Variant Management)	20
3.3.6	FR-INV: Sổ cái & Quản lý tồn kho (Inventory Ledger & Stock Management)	21
3.3.7	FR-COST: Tính giá vốn bình quân gia quyền (Weighted Average Cost Calculation)	24
3.3.8	FR-ORD: Trung tâm đơn hàng tập trung (Centralized Order Hub)	24
3.3.9	FR-RSE: Cơ chế chống bán vượt tồn (Anti-Oversell Mechanism)	25
3.3.10	FR-POS: Bán hàng tại quầy — PWA POS (Counter Sales)	26
3.3.11	FR-REP: Báo cáo & Cảnh báo (Reports & Alerts)	27
3.3.12	FR-SIM: Webhook Simulator & Xử lý thời gian thực	28
3.3.13	External Interfaces	28
3.3.14	Quality Attributes	28
3.3.15	Business Rules	29
3.3.16	Common Requirements	30
3.3.17	Application Messages List	30
3.3.18	Bảng tóm tắt yêu cầu theo nhóm chức năng	30
3.3.19	Kết luận đặc tả yêu cầu	31
3.4	Tổng hợp yêu cầu chức năng và phi chức năng	31
3.5	Mức độ sẵn sàng cho giai đoạn thiết kế cơ sở dữ liệu	32
3.6	Nhận định cuối cùng	32
4	Triển khai Cơ sở dữ liệu và Kiểm thử	33
4.1	Môi trường và công nghệ sử dụng	33
4.1.1	Công nghệ đã sử dụng trong phạm vi bàn giao (Thiết kế + Database)	33
4.1.2	Công nghệ đã hoạch định cho hệ thống hoàn chỉnh (chưa triển khai trong học kỳ này)	33
4.1.3	Môi trường phát triển	34
4.2	Thiết kế Logic và Vật lý Cơ sở dữ liệu	34
4.2.1	Danh mục thực thể (Entity Catalog)	34
4.2.2	Chứng minh Thiết kế Logic (Logical Design Correctness)	35
4.2.3	Script DDL (Định nghĩa bảng)	36
4.2.4	Cơ chế Append-only cho Sổ cái Tồn kho	39
4.2.5	Cô lập Đa khách thuê — Tenant Isolation	40
4.2.6	Kiểm soát đồng thời — Chống bán vượt tồn (Inventory Reservation)	40
4.2.7	Giá vốn bình quân (WAC) và Snapshot giá vốn đơn hàng	41
4.2.8	Chỉ mục (Index) theo Workload	41
4.2.9	Chiến lược Backup / Recovery	41
4.2.10	Đánh giá OLTP (Online Transaction Processing)	42
4.2.11	Đánh giá Partitioning (đánh giá, chưa triển khai)	42
4.2.12	Dữ liệu mẫu (Seed Data)	42
4.3	Kiểm thử Cơ sở dữ liệu	42
4.3.1	Phạm vi kiểm thử	42
4.3.2	Kiểm chứng ràng buộc Append-only	43
4.3.3	Kiểm chứng cô lập dữ liệu đa Tenant	43
4.3.4	Kiểm chứng tính toàn vẹn Ledger ↔ StockBalance	44
4.3.5	Kiểm chứng tổng tiền đơn hàng (Orders.TotalAmount)	44
4.3.6	Kiểm chứng công thức giá vốn bình quân	44
4.3.7	Mô phỏng Concurrency (kịch bản Flash Sale)	45
4.3.8	Kết quả kiểm thử	45
4.3.9	Lỗi phát hiện và hướng xử lý	46
5	Triển khai, Đánh giá và Hướng phát triển	47

5.1	Triển khai hệ thống	47
5.1.1	Phạm vi triển khai thực tế	47
5.1.2	Kiến trúc triển khai Database	47
5.1.3	Hướng dẫn cài đặt	47
5.1.4	Kiến trúc triển khai đã hoạch định cho hệ thống hoàn chỉnh	47
5.2	Đánh giá kết quả đạt được	48
5.2.1	Kết quả đạt được so với kế hoạch	48
5.2.2	Đánh giá theo yêu cầu phi chức năng	48
5.2.3	Bài học kinh nghiệm	48
5.3	Hạn chế và hướng phát triển	49
5.3.1	Hạn chế hiện tại	49
5.3.2	Hướng phát triển	49

Danh sách hình vẽ

3.1	Sơ đồ kiến trúc tổng thể hệ thống (Architecture / System Context Diagram)	14
3.2	Sơ đồ Use Case tổng quát của hệ thống	15
3.3	Sơ đồ ERD toàn bộ cơ sở dữ liệu (20 thực thể)	18
3.4	UML Class Diagram các entity chính	19
3.5	Sequence Diagram — luồng nhập hàng & tính giá vốn bình quân	22
3.6	Sequence Diagram — luồng chuyển kho 2 bước (Outbound/Inbound)	23
3.7	Sequence Diagram — luồng kiểm kho & sinh bút toán điều chỉnh	23
3.8	State Diagram — vòng đời đơn hàng (Order State Machine)	25
3.9	Sequence Diagram — luồng giữ chỗ tồn kho (Stock Reservation)	26
3.10	Sequence Diagram — luồng thanh toán nhanh tại POS (Fast Checkout)	27

Danh sách bảng

1.1	Thông tin thành viên nhóm	1
2.1	Bảng phân rã công việc và ước lượng nỗ lực	6
2.2	Danh sách rủi ro và phương án xử lý	7
2.3	Kế hoạch onboard công cụ/công nghệ	8
2.4	Kế hoạch bàn giao theo tuần (5 tuần)	8
2.5	Danh sách 14 hạng mục bàn giao cuối kỳ	9
2.6	Bảng phân công trách nhiệm	10
2.7	Kế hoạch giao tiếp trong dự án	11
2.8	Công cụ và hạ tầng sử dụng trong dự án	11
3.1	Bảng phân tích thành phần hệ thống theo mô đun chức năng	14
3.2	Danh sách Actor của hệ thống	15
3.3	Danh sách và mô tả Use Case	16
3.4	Danh sách màn hình dự kiến (chưa thiết kế mockup chi tiết)	17
3.5	Ma trận phân quyền theo vai trò (RBAC)	17
3.6	Các chức năng không có màn hình (chạy nền/hệ thống)	18
3.7	Tóm tắt các FR còn lại — nhóm FR-AUTH	20
3.8	Tóm tắt các FR còn lại — nhóm FR-PROD	21
3.9	Tóm tắt các FR còn lại — nhóm FR-POS	27
3.10	Tóm tắt chức năng — nhóm FR-REP	28
3.11	Tóm tắt chức năng — nhóm FR-SIM	28
3.12	Danh sách quy tắc nghiệp vụ (Business Rules)	29
3.13	Danh sách thông báo ứng dụng	30
3.14	Bảng tóm tắt yêu cầu theo chức năng chính	31
3.15	Bảng tổng hợp yêu cầu chức năng và phi chức năng theo ưu tiên	32
4.1	Danh mục 20 thực thể triển khai trong schema SQL Server	35
4.2	Chiến lược backup/recovery đề xuất	41
4.3	Tổng hợp kết quả kiểm thử mức Database	45
5.1	Đối chiếu kế hoạch và kết quả thực hiện	48

Chương 1

Giới thiệu tổng quan dự án

1.1 Tổng quan dự án

1.1.1 Thông tin dự án

- **Tên dự án (Tiếng Anh):** Omnichannel Inventory and Sales Management System (OISM).
- **Tên dự án (Tiếng Việt):** Hệ thống quản lý bán hàng và tồn kho đa kênh.
- **Loại phần mềm:** Nền tảng SaaS đa khách thuê (multi-tenant), gồm Backend API, Admin Dashboard (Web) và ứng dụng POS dạng Progressive Web App (PWA).
- **Mục đích:** Xây dựng một hệ thống giúp doanh nghiệp bán lẻ quản lý tập trung tồn kho và đơn hàng đến từ nhiều kênh bán (cửa hàng vật lý, Shopee, TikTok Shop, Lazada), loại bỏ tình trạng bán vượt tồn kho (oversell), sai lệch giá vốn và mất khả năng truy vết dữ liệu.
- **Phạm vi bàn giao của nhóm:** Thiết kế hệ thống toàn diện (kiến trúc, đặc tả yêu cầu, mô hình dữ liệu, luồng nghiệp vụ, đặc tả API) và xây dựng cơ sở dữ liệu vật lý (SQL Server) kèm dữ liệu mẫu minh họa. Việc lập trình Backend API, Frontend, tích hợp AI, CI/CD và triển khai thực tế nằm ngoài phạm vi bàn giao của học kỳ này (xem chi tiết tại mục 1.6).

1.1.2 Thành viên nhóm

Bảng 1.1: Thông tin thành viên nhóm

Họ và tên	Vai trò	Email	MSSV
(Điền tên GVHD)	Giảng viên hướng dẫn	(điền email)	–
(Điền tên)	Trưởng nhóm / Data Architect (TV1)	(điền email)	(điền MSSV)
(Điền tên)	Database Engineer (TV2)	(điền email)	(điền MSSV)
(Điền tên)	Solution Architect (TV3)	(điền email)	(điền MSSV)
(Điền tên)	QA & Project Documentation Lead (TV4)	(điền email)	(điền MSSV)

Bảng thành viên trên đang để placeholder — điền tên thật, email, MSSV và vai trò cụ thể của từng người trước khi nộp báo cáo.

1.2 Bối cảnh và nền tảng đề tài

Trong bối cảnh bán lẻ hiện đại, doanh nghiệp ngày càng vận hành song song nhiều kênh bán hàng: cửa hàng vật lý (Point-of-Sale), và các sàn thương mại điện tử như Shopee, TikTok Shop, Lazada. Việc quản lý tồn kho và đơn hàng trên các kênh phân mảnh này đặt ra nhiều thách thức nghiêm trọng:

- **Bán vượt tồn kho (oversell):** Khi tồn kho không được đồng bộ tức thời giữa các kênh, cùng một sản phẩm có thể được bán cho nhiều khách hàng khác nhau trên các kênh khác nhau vượt quá số lượng thực tế còn trong kho.
- **Giá vốn hàng bán (COGS) không chính xác:** Khi nhập hàng nhiều đợt với đơn giá khác nhau, việc tính giá vốn thủ công hoặc tính sai dẫn đến báo cáo lợi nhuận sai lệch.
- **Sai lệch dữ liệu tồn kho:** Cập nhật tồn kho thủ công qua nhiều đầu mối (nhân viên kho, nhân viên bán hàng, đối tác sàn TMĐT) dễ dẫn đến sai số cộng dồn theo thời gian, và không có khả năng truy vết lại lịch sử thay đổi khi cần kiểm toán.

Để giải quyết các vấn đề trên, doanh nghiệp bán lẻ cần một giải pháp phần mềm tích hợp, có khả năng xử lý đồng thời (concurrency) ở mức cao, đảm bảo tính toàn vẹn của sổ sách tồn kho theo thời gian thực, và hỗ trợ mô hình SaaS đa khách thuê (multi-tenant) để một nền tảng có thể phục vụ nhiều cửa hàng độc lập với dữ liệu cô lập tuyệt đối.

1.3 Hệ thống hiện tại

1.3.1 Phần mềm quản lý bán hàng đơn kênh truyền thống

Mô tả: Phần lớn cửa hàng nhỏ và vừa hiện dùng phần mềm quản lý bán hàng (ví dụ Sapo, KiotViet, hoặc Excel thủ công) chỉ tập trung vào một kênh bán chính, thường là cửa hàng vật lý hoặc một sàn TMĐT duy nhất.

Nhược điểm:

- Không đồng bộ tồn kho theo thời gian thực giữa nhiều kênh bán khác nhau, dẫn đến tình trạng oversell khi vận hành đa kênh.
- Không có cơ chế sổ cái (ledger) ghi nhận từng biến động tồn kho — chỉ lưu số dư cuối cùng, mất khả năng truy vết và kiểm toán khi có sai lệch.
- Tính giá vốn theo phương pháp đơn giản (giá nhập gần nhất hoặc giá cố định), không phản ánh đúng giá vốn bình quân khi nhập hàng nhiều đợt với giá khác nhau.

1.3.2 Giải pháp ERP/Middleware kết nối đa kênh của bên thứ ba

Mô tả: Một số nền tảng middleware (ví dụ Sapo Omnichannel, Haravan) cho phép kết nối nhiều kênh bán vào một kho trung tâm.

Nhược điểm:

- Chi phí thuê bao cao, khó tùy biến sâu theo quy trình nghiệp vụ đặc thù của từng doanh nghiệp (ví dụ công thức tính giá vốn riêng, quy tắc giữ chỗ tồn kho riêng).
- Cơ chế chống bán vượt tồn (nếu có) thường xử lý bất đồng bộ theo chu kỳ (polling), có độ trễ vài giây đến vài phút — không đủ chặt chẽ trong kịch bản flash sale với nhiều đơn hàng tranh chấp cùng lúc trên 1 sản phẩm còn ít tồn kho.
- Không phải nền tảng mã nguồn mở/tự triển khai, nên doanh nghiệp không kiểm soát được hạ tầng dữ liệu và khó tích hợp sâu với hệ thống kế toán nội bộ.

1.4 Cơ hội kinh doanh và vận hành

- Việc tập trung quản lý tồn kho và đơn hàng đa kênh trên một nền tảng duy nhất giúp doanh nghiệp bán lẻ loại bỏ hoàn toàn rủi ro oversell — vấn đề trực tiếp ảnh hưởng đến uy tín và trải nghiệm khách hàng trên các sàn TMĐT.
- Cơ chế sổ cái tồn kho append-only (Inventory Ledger) tạo ra một nguồn dữ liệu đáng tin cậy duy nhất (single source of truth), giúp doanh nghiệp kiểm toán chính xác, phát hiện gian lận hoặc sai sót vận hành nhanh chóng.
- Mô hình SaaS đa khách thuê (multi-tenant) cho phép một nền tảng phục vụ đồng thời nhiều cửa hàng độc lập, giảm chi phí hạ tầng và mở ra khả năng thương mại hóa sản phẩm sau khi hoàn thiện.
- Cơ chế tính giá vốn bình quân gia quyền tự động giúp doanh nghiệp có báo cáo lợi nhuận gộp chính xác theo thời gian thực, hỗ trợ ra quyết định kinh doanh nhanh và đúng.

1.5 Tầm nhìn sản phẩm

- Tầm nhìn của OISM là trở thành nền tảng quản lý bán hàng và tồn kho đa kênh đáng tin cậy, nơi mọi biến động tồn kho — dù đến từ kênh nào — đều được ghi nhận minh bạch, không thể sửa đổi sau khi phát sinh, và luôn phản ánh đúng số lượng khả dụng thực tế.
- Hệ thống hướng đến kiến trúc SaaS đa khách thuê (multi-tenant) trên nền Clean Architecture, cho phép mở rộng phục vụ nhiều doanh nghiệp bán lẻ với dữ liệu cô lập tuyệt đối và hiệu năng ổn định ngay cả khi có hàng chục đơn hàng tranh chấp đồng thời trên cùng một sản phẩm (kịch bản flash sale).
- Trong phiên bản hoàn chỉnh, OISM cung cấp ba điểm chạm chính: Admin Dashboard cho chủ cửa hàng/quản lý, ứng dụng POS dạng PWA cho nhân viên bán hàng tại quầy, và Webhook Simulator để mô phỏng luồng đơn hàng từ các sàn TMĐT.

1.6 Phạm vi và giới hạn của dự án

1.6.1 Các tính năng chính (theo thiết kế đầy đủ của hệ thống)

1.6.1.1 Tính năng dành cho Owner (Chủ cửa hàng)

- **FE-01:** Đăng ký Tenant mới, quản lý chi nhánh/kho hàng.
- **FE-02:** Quản lý người dùng và phân quyền (Owner/Staff/Cashier).
- **FE-03:** Quản lý danh mục, thương hiệu, sản phẩm/biến thể (SKU), giá bán.
- **FE-04:** Xem toàn bộ báo cáo doanh thu, giá vốn, lợi nhuận gộp.

1.6.1.2 Tính năng dành cho Staff (Nhân viên quản lý kho/đơn hàng)

- **FE-01:** Quản lý phiếu nhập hàng (Purchase Receipt) và giá vốn bình quân.
- **FE-02:** Quản lý chuyển kho giữa các chi nhánh (2 bước xuất/nhập).
- **FE-03:** Quản lý phiên kiểm kho (Stocktake) và bút toán điều chỉnh.
- **FE-04:** Xử lý đơn hàng: duyệt, in phiếu giao hàng, hủy đơn.

1.6.1.3 Tính năng dành cho Cashier (Thu ngân tại quầy — POS)

- **FE-01:** Tìm kiếm sản phẩm nhanh theo Tên/SKU/Barcode.
- **FE-02:** Kiểm tra tồn khả dụng tức thời, chặn bán vượt tồn.
- **FE-03:** Thanh toán (tiền mặt/QR) và in hóa đơn.
- **FE-04:** Thực hiện checkout nhanh (Reserve → Confirm → Deduct trong 1 giao dịch atomic).

1.6.1.4 Tính năng hệ thống (chạy nền/tích hợp)

- **FE-01:** Webhook Simulator mô phỏng đơn hàng từ Shopee/TikTok Shop/Lazada.
- **FE-02:** Thông báo real-time (SignalR) cho đơn hàng mới.
- **FE-03:** Tự động hủy đơn Reserved quá hạn, tổng hợp báo cáo hằng ngày (Hangfire).
- **FE-04:** Cảnh báo tồn kho thấp khi available chạm ngưỡng tối thiểu.

1.6.2 Phạm vi bàn giao thực tế của nhóm trong học kỳ này

Dự án OISM có quy mô kỹ thuật lớn, bao trùm cả Backend API (.NET 8), Frontend (Admin Dashboard + POS PWA bằng ReactJS), tích hợp AI dự báo nhập hàng và triển khai CI/CD. Do giới hạn thời gian và nguồn lực của học kỳ, nhóm quyết định thu hẹp phạm vi bàn giao thực tế xuống còn hai hạng mục cốt lõi, làm nền tảng vững chắc cho các học kỳ/giai đoạn phát triển tiếp theo:

1. **Thiết kế hệ thống toàn diện:** SRS đầy đủ (yêu cầu chức năng, phi chức năng), kiến trúc Clean Architecture, ERD, UML Class Diagram, Sequence Diagram cho các luồng nghiệp vụ trọng yếu, đặc tả API (Swagger/Postman).
2. **Cơ sở dữ liệu:** Script DDL SQL Server đầy đủ, cơ chế ràng buộc append-only cho sổ cái tồn kho, index tối ưu, dữ liệu mẫu (seed data) minh họa, và script kiểm tra tính toàn vẹn dữ liệu.

Không nằm trong phạm vi bàn giao của học kỳ này: Lập trình Backend API (.NET), lập trình Frontend (Admin Dashboard, POS PWA), tích hợp AI dự báo nhập hàng, tích hợp SignalR/Hangfire thực tế, CI/CD, và triển khai (deploy) hệ thống hoàn chỉnh lên môi trường production.

1.6.3 Hạn chế và loại trừ

- **LI-1: Chưa có phần mềm chạy được đầu-cuối (end-to-end):** Vì Backend/Frontend chưa được lập trình, các chức năng ở mục 1.6.1 chỉ tồn tại ở dạng đặc tả thiết kế, chưa thể demo trực tiếp trên giao diện.
- **LI-2: Webhook Simulator và tích hợp sàn TMĐT thật:** Trong phạm vi đồ án, các sàn Shopee/TikTok Shop/Lazada chỉ được mô phỏng qua thiết kế Webhook Simulator, không kết nối API thật của các sàn.
- **LI-3: Kiểm thử concurrency (NFR-PERF-02):** Do chưa có Backend thật, kịch bản 50 đơn hàng đồng thời tranh 1 SKU chỉ được kiểm chứng ở mức thiết kế (script SQL mô phỏng transaction/locking), chưa đo lường được trên hệ thống chạy thật.

Chương 2

Kế hoạch Quản lý Dự án

2.1 Tổng quan

2.1.1 Phạm vi và Ước lượng

Dự án được chia nhỏ theo cấu trúc phân rã công việc (WBS), ước lượng nỗ lực theo ngày công (man-day) cho từng hạng mục, giới hạn trong phạm vi **Thiết kế hệ thống + Cơ sở dữ liệu** đã nêu ở mục 1.6.

2.1.2 Mục tiêu dự án

- **Thời hạn:** Hoàn thành trong 5 tuần, trước thời điểm báo cáo tiến độ cuối kỳ với GVHD.
- **Nỗ lực phân bổ (man-day):** Khoảng 64 ngày công cho 4 thành viên (trung bình 16 ngày công/người).
- **Tiêu chí hoàn thành:** Toàn bộ 14 hạng mục tài liệu/mã nguồn liệt kê ở mục 2.3 được bàn giao đầy đủ, có thể chạy được (đối với phần Database) hoặc thẩm định được (đối với phần thiết kế).

2.1.3 Rủi ro dự án

Bảng 2.1: Bảng phân rã công việc và ước lượng nỗ lực

#	Hạng mục (WBS)	Độ phức tạp	Ước lượng (ngày công)
1	Khởi tạo (đọc đề cương, xác định phạm vi thu hẹp)	Đơn giản	3
2	Soạn thảo SRS (yêu cầu chức năng & phi chức năng)	Trung bình	6
3.1	Thiết kế ERD (Entity Relationship Diagram)	Phức tạp	5
3.2	Viết script DDL SQL Server (bảng, khóa, ràng buộc)	Phức tạp	5
3.3	Thiết kế trigger/rule chặn UPDATE/DELETE trên Ledger	Trung bình	3
3.4	Thiết kế index tối ưu đa khách thuê	Đơn giản	2
3.5	Thiết kế kiến trúc tổng thể (Architecture Diagram)	Trung bình	3
3.6	Vẽ Use Case Diagram & UML Class Diagram	Trung bình	4
3.7	Vẽ Sequence Diagram (6 luồng nghiệp vụ trọng yếu)	Phức tạp	6
3.8	Đặc tả API (Swagger/Postman)	Trung bình	4
4.1	Cài đặt SQL Server (Docker), chạy schema thật	Đơn giản	2
4.2	Viết seed data (tenant, user, sản phẩm, tồn kho, đơn hàng mẫu)	Trung bình	4
4.3	Viết script kiểm tra toàn vẹn dữ liệu	Trung bình	3
5.1	RTM đầy đủ & Data Dictionary	Trung bình	4
5.2	Test Plan / Test Case (thiết kế, không chạy)	Trung bình	3
5.3	Quản lý GitHub (repo, Issues, Milestones)	Đơn giản	2
5.4	Tổng hợp báo cáo, README, chuẩn bị bảo vệ	Trung bình	5
Tổng nỗ lực ước tính (ngày công)			64

Bảng 2.2: Danh sách rủi ro và phương án xử lý

#	Mô tả rủi ro	Ảnh hưởng	Khả năng	Phương án xử lý
1	Thiết kế ERD sai ngay từ đầu, phát hiện muộn khi đã viết DDL	Phải làm lại schema và seed data, trễ tiến độ	Trung bình	TV1 hoàn thành ERD trước, review chéo với TV3 (Class Diagram) trước khi viết DDL
2	Cơ chế Anti-Oversell (locking) thiết kế sai logic, không chống được race-condition	Sai lệch mục tiêu kỹ thuật cốt lõi của đề tài	Trung bình	Thiết kế Sequence Diagram chi tiết, review kỹ trước khi đưa vào SRS chính thức
3	Thành viên vô tình mở rộng phạm vi sang code Backend/Frontend	Lãng phí thời gian ngoài phạm vi bàn giao đã thống nhất	Thấp	Nhắc lại phạm vi (mục 1.6) đầu mỗi buổi họp nhóm
4	Trigger chặn UPDATE/DELETE trên Ledger viết sai, không raise lỗi đúng cách	Không đáp ứng NFR-SEC-03, ảnh hưởng điểm thiết kế	Thấp	Viết kèm script test tự kiểm chứng (thử UPDATE/DELETE, xác nhận bị chặn)
5	Tài liệu SRS/RTM không đồng bộ khi có thay đổi thiết kế giữa chừng	Tài liệu bàn giao không nhất quán	Trung bình	TV4 (QA lead) rà soát chéo cuối mỗi tuần, cập nhật RTM song song với thiết kế

2.2 Phương pháp Quản lý

2.2.1 Quy trình dự án

Nhóm áp dụng mô hình **Phát triển lặp và tăng trưởng (Iterative and Incremental Development)**, chia thành 5 giai đoạn tương ứng 5 tuần, mỗi giai đoạn có sản phẩm đầu ra rõ ràng, hỗ trợ review chéo giữa các thành viên trước khi chuyển giai đoạn tiếp theo.

Lý do lựa chọn mô hình này:

- Thiết kế dữ liệu (ERD, DDL) phải hoàn thành trước và đúng, vì mọi công việc còn lại (Class Diagram, seed data, script kiểm tra) đều phụ thuộc vào nó.
- Cho phép phát hiện sớm các lỗi thiết kế nghiệp vụ (ví dụ: công thức giá vốn, cơ chế locking) trước khi lan sang các tài liệu khác.
- Dễ tích hợp phản hồi của GVHD sau mỗi buổi báo cáo tiến độ hằng tuần.

2.2.2 Quản lý chất lượng

Phòng ngừa lỗi: Mọi thiết kế trọng yếu (ERD, cơ chế Anti-Oversell) phải được ít nhất một thành viên khác review trước khi đưa vào tài liệu chính thức.

Review: TV4 (QA & Documentation Lead) chịu trách nhiệm rà soát tính nhất quán giữa các tài liệu (SRS, ERD, DDL, RTM) mỗi tuần.

Kiểm thử thiết kế: Vì không có Backend thật, "kiểm thử" trong phạm vi đồ án là kiểm thử ở mức Database — chạy thử script DDL, seed data, và script kiểm tra toàn vẹn dữ liệu (đối chiếu Ledger với StockBalance, kiểm chứng công thức giá vốn bình quân bằng dữ liệu mẫu thực tế).

2.2.3 Kế hoạch onboard công nghệ

Do các thành viên đã có nền tảng từ các học phần trước, thời gian onboard được rút gọn còn 2 ngày đầu Tuần 1.

Bảng 2.3: Kế hoạch onboard công cụ/công nghệ

Nội dung	Thành viên tham gia	Thời gian	Yêu cầu
SQL Server nâng cao (trigger, lock hints)	TV1, TV2	Ngày 1	Bắt buộc
Công cụ vẽ ERD/UML (dbdiagram.io, draw.io)	TV1, TV3	Ngày 1	Bắt buộc
Git/GitHub: Issues, Milestones, Board	Toàn bộ thành viên	Ngày 1	Bắt buộc
Chuẩn viết SRS/RTM (IEEE 830 rút gọn)	TV4	Ngày 2	Bắt buộc

2.3 Sản phẩm Bàn giao

Với thời hạn 5 tuần, dự án được chia thành 5 giai đoạn, mỗi giai đoạn tương ứng đúng 1 tuần.

Bảng 2.4: Kế hoạch bàn giao theo tuần (5 tuần)

#	Giai đoạn	Thời gian	Nội dung
1	Giai đoạn 1	Tuần 1	TV1 hoàn thành ERD, DDL nháp; TV4 setup GitHub (repo, Issues, Milestones)
2	Giai đoạn 2	Tuần 2	TV1 hoàn thiện DDL/trigger append-only; TV2 bắt đầu setup Database thật; TV3 vẽ Architecture Diagram + Use Case Diagram
3	Giai đoạn 3	Tuần 3	TV2 viết seed data; TV3 vẽ Class Diagram + Sequence Diagram (6 luồng nghiệp vụ)
4	Giai đoạn 4	Tuần 4	TV2 viết script kiểm tra toàn vẹn dữ liệu; TV3 hoàn thiện đặc tả API; TV4 viết Test Plan + RTM đầy đủ
5	Giai đoạn 5	Tuần 5	Cả nhóm review chéo toàn bộ tài liệu; TV4 tổng hợp README, Data Dictionary và báo cáo cuối kỳ

Bảng 2.5: Danh sách 14 hạng mục bàn giao cuối kỳ

#	Hạng mục	Phụ trách chính
1	SRS (Software Requirements Specification)	TV4 (tổng hợp)
2	Architecture Design Document	TV3
3	ERD (Entity Relationship Diagram)	TV1
4	UML Class Diagram	TV3
5	Use Case Diagram	TV3
6	Sequence Diagram (6 luồng nghiệp vụ)	TV3
7	Đặc tả API (Swagger/Postman)	TV3
8	Script DDL SQL Server (schema.sql)	TV1
9	Trigger/rule chặn UPDATE/DELETE Ledger	TV1
10	Seed data (dữ liệu mẫu)	TV2
11	Script kiểm tra toàn vẹn dữ liệu	TV2
12	Data Dictionary	TV1
13	Test Plan / Test Case	TV4
14	README + hướng dẫn cài đặt (Installation Guide)	TV4 / TV2

2.4 Phân công Trách nhiệm

Ký hiệu: D = Thực hiện chính; R = Kiểm tra/Review; S = Hỗ trợ; I = Được thông báo.

Bảng 2.6: Bảng phân công trách nhiệm

Trách nhiệm	TV1 (Data Architect)	TV2 (DB Engineer)	TV3 (Solution Architect)	TV4 (QA & Docs)
Lập kế hoạch và theo dõi tiến độ	S	S	S	D
ERD, DDL, trigger, Data Dictionary	D	S	R	I
Tạo Database thật, seed data, script kiểm tra toàn vẹn	S	D	I	I
Architecture/Use Case/Class/Sequence Diagram, API Spec	S	I	D	R
RTM, Test Plan, GitHub, README, báo cáo cuối	I	I	S	D
Soạn thảo Chương 1 – Giới thiệu dự án	S	S	S	D
Soạn thảo Chương 2 – Kế hoạch quản lý	S	S	S	D
Soạn thảo Chương 3 – SRS	D	S	D	R
Soạn thảo Chương 4 – Triển khai CSDL & Kiểm thử	S	D	I	R
Soạn thảo Chương 5 – Đánh giá & hướng phát triển	S	S	S	D

2.5 Giao tiếp Dự án

Bảng 2.7: Kế hoạch giao tiếp trong dự án

Hoạt động	Đối tượng	Mục đích	Tần suất	Hình thức
Họp nhóm hàng tuần	Toàn bộ thành viên	Rà soát tiến độ 5 hạng mục, review chéo tài liệu	1 lần/tuần	Trực tiếp/Google Meet
Báo cáo tiến độ với GVHD	GVHD, nhóm	Xin ý kiến về thiết kế (ERD, cơ chế Anti-Oversell), giải đáp thắc mắc	1 lần/tuần	Google Meet
Cập nhật GitHub Issues	Toàn bộ thành viên	Theo dõi trạng thái từng hạng mục bàn giao	Hàng ngày	GitHub Projects
Họp đột xuất	Toàn bộ thành viên	Giải quyết bất đồng thiết kế (ví dụ tranh luận về cơ chế locking)	Khi cần	Google Meet

2.6 Quản lý Cấu hình

2.6.1 Quản lý tài liệu

Công cụ tài liệu: Microsoft Word (SRS, báo cáo), draw.io/dbdiagram.io (ERD, UML), Google Docs/Sheets cho biên bản họp.
Lưu trữ và quản lý file: Google Drive (bản nháp) và GitHub repo (bản chính thức, có version).

2.6.2 Quản lý mã nguồn & tài liệu thiết kế

Toàn bộ script SQL (DDL, seed data, script kiểm tra toàn vẹn) và tài liệu thiết kế (SRS, Data Dictionary) được quản lý bằng Git trên GitHub, tuân theo quy ước nhánh: main, feature/erd, feature/ddl-schema, feature/seed-data, report. Mỗi thay đổi schema quan trọng được tạo thành một Pull Request riêng để TV4 review trước khi merge vào main.
Nhóm áp dụng quy trình làm việc Git cơ bản: mỗi thành viên làm việc trên nhánh riêng, dùng git add/git commit/git push để cập nhật, và git pull trước khi bắt đầu công việc mới để tránh xung đột (conflict). Tiến độ dự án được theo dõi qua GitHub Issues, Milestones và Issue Board (kéo-thả theo trạng thái To Do/Doing/Done).

2.6.3 Công cụ và hạ tầng

Bảng 2.8: Công cụ và hạ tầng sử dụng trong dự án

Hạng mục	Công cụ / Hạ tầng
Cơ sở dữ liệu	SQL Server 2022 (chạy qua Docker)
Vẽ ERD/UML	dbdiagram.io, draw.io
Đặc tả API	Swagger Editor / Postman
Quản lý mã nguồn & tài liệu	GitHub (repo, Issues, Milestones, Board)
Soạn thảo tài liệu	Microsoft Word, LaTeX (báo cáo cuối kỳ)
Quản lý dự án	GitHub Projects, Google Sheet (theo dõi WBS)

Lưu ý: Backend (.NET 8), Frontend (ReactJS), Docker Compose triển khai đầy đủ, và CI/CD (GitHub Actions) là công nghệ đã hoạch định cho hệ thống hoàn chỉnh (xem Chương 3), nhưng không nằm trong phạm vi triển khai thực tế của học kỳ này.

Chương 3

Software Requirement Specification

Mục tiêu của Chương 3: Chương này trình bày đặc tả yêu cầu phần mềm của Hệ thống quản lý bán hàng và tồn kho đa kênh (OISM) dưới góc nhìn người dùng, chức năng, dữ liệu, giao diện và yêu cầu phi chức năng. Đây là chương trọng tâm của đồ án, vì phạm vi bàn giao thực tế của nhóm (mục 1.6) tập trung vào thiết kế — nội dung chương này chính là sản phẩm chi tiết nhất mà nhóm hoàn thành.

3.1 Phạm vi của hệ thống

Hệ thống được thiết kế để phục vụ doanh nghiệp bán lẻ vận hành đồng thời nhiều kênh bán (cửa hàng vật lý và các sàn TMĐT), với ba vấn đề nghiệp vụ cốt lõi cần giải quyết: chống bán vượt tồn kho (oversell), tính giá vốn bình quân gia quyền chính xác, và đảm bảo khả năng truy vết/kiểm toán tuyệt đối của mọi biến động tồn kho thông qua cơ chế sổ cái append-only.

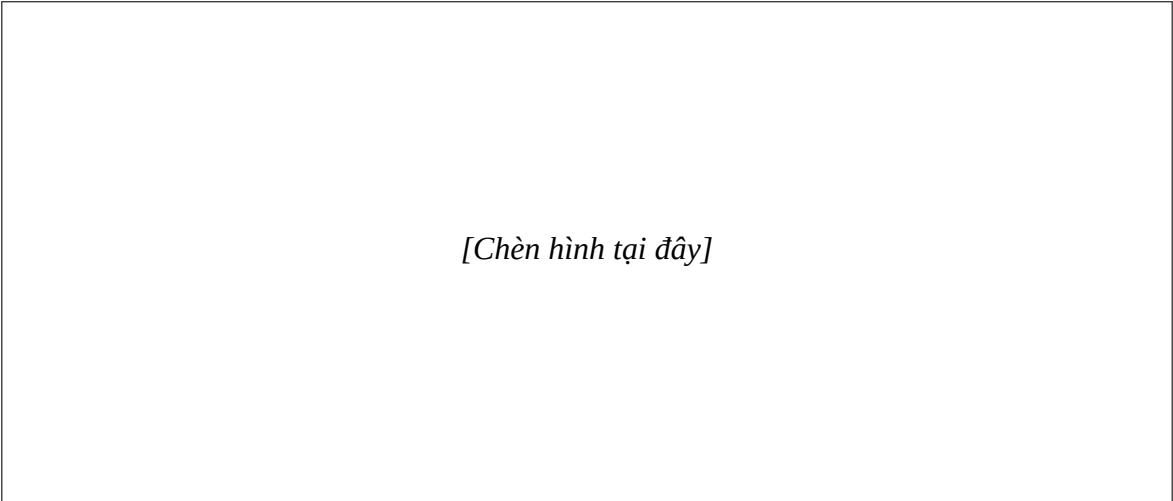
Trong phiên bản thiết kế đầu tiên (phạm vi đồ án), hệ thống tập trung đặc tả đầy đủ 9 nhóm chức năng cốt lõi: Xác thực đa khách thuê, Quản lý sản phẩm/biến thể, Sổ cái & quản lý tồn kho, Tính giá vốn bình quân, Trung tâm đơn hàng tập trung, Cơ chế chống bán vượt tồn, Bán hàng tại quầy (POS), Báo cáo & cảnh báo, và Webhook Simulator & xử lý thời gian thực. Các tính năng nâng cao như tích hợp AI dự báo nhập hàng và định vị nâng cao được ghi nhận trong thiết kế nhưng dời sang giai đoạn phát triển tiếp theo.

3.2 Product Overview

Omnichannel Inventory and Sales Management System (OISM) là một nền tảng SaaS đa khách thuê (multi-tenant), thay thế các quy trình quản lý tồn kho rời rạc hiện tại của doanh nghiệp bán lẻ đa kênh. Hệ thống bao gồm quản lý sản phẩm/biến thể, sổ cái tồn kho append-only, tính giá vốn bình quân gia quyền tự động, trung tâm tiếp nhận đơn hàng đa kênh, cơ chế chống bán vượt tồn, bán hàng tại quầy (POS) và báo cáo doanh thu/lợi nhuận.

Hệ thống được thiết kế theo Clean Architecture, gồm ba thành phần chính: Backend API (.NET 8) xử lý toàn bộ nghiệp vụ và dữ liệu, Admin Dashboard (ReactJS) dành cho Owner/Staff quản trị, và POS Application (ReactJS PWA) dành cho Cashier bán hàng tại quầy. Ba thành phần giao tiếp qua REST API và kênh real-time (SignalR).

Sơ đồ kiến trúc dưới đây minh họa các lớp và thành phần của hệ thống cho phiên bản thiết kế 1.0. Hệ thống dự kiến mở rộng ở các học kỳ sau, bổ sung tích hợp AI dự báo nhập hàng và kết nối API thật của các sàn TMĐT.



Hình 3.1: Sơ đồ kiến trúc tổng thể hệ thống (Architecture / System Context Diagram)

Hình 1 — Cần bổ sung hình: *Hình_01_KienTrucTongThe.png* — Sơ đồ kiến trúc tổng thể, 4 lớp Clean Architecture (Domain/Application/Infrastructure/Presentation) và luồng giao tiếp giữa Backend API, Admin Dashboard, POS App. Thay khối khung nét đứt phía trên bằng `\includegraphics` khi có file hình.

3.2.1 Mô tả luồng dữ liệu và ràng buộc nghiệp vụ

Hệ thống hoạt động theo mô hình sổ cái là nguồn sự thật duy nhất (ledger as single source of truth): mọi thay đổi tồn kho — dù đến từ nhập hàng, bán hàng, chuyển kho hay kiểm kho — đều được ghi thành một dòng mới trong bảng InventoryTransaction trước khi sổ dư tồn kho hiển thị (StockBalance) được cập nhật. Cách tiếp cận này giúp đảm bảo khả năng truy vết và kiểm toán tuyệt đối, khác biệt với cách các hệ thống bán lẻ truyền thống chỉ lưu sổ dư cuối cùng.

Mô đun	Vai trò	Mô tả trách nhiệm
Admin Dashboard	Owner/Staff	Quản lý sản phẩm, chi nhánh, xem sổ cái tồn kho, xử lý đơn hàng, xem báo cáo lợi nhuận
POS Application	Cashier	Bán hàng nhanh tại quầy, quét barcode, thanh toán, in hóa đơn
Backend API (Domain/Application)	Logic nghiệp vụ	Xử lý reserve/confirm/deduct, tính giá vốn bình quân, state machine đơn hàng
Inventory Ledger Engine	Sổ cái	Ghi nhận append-only mọi biến động tồn kho, tuyệt đối không cho sửa/xóa
Anti-Oversell Engine	Chống bán vượt tồn	Khóa hàng (locking), tính available theo thời gian thực
Webhook Simulator	Tích hợp	Mô phỏng đơn hàng từ Shopee/TikTok Shop/Lazada
Reporting Service	Báo cáo	Tổng hợp doanh thu, giá vốn, lợi nhuận gộp, cảnh báo tồn thấp

Bảng 3.1: Bảng phân tích thành phần hệ thống theo mô đun chức năng

3.3 User Requirements

3.3.1 Actors

#	Actor	Mô tả
1	Owner (Chủ cửa hàng)	Quản trị viên cao nhất của Tenant: quản lý chi nhánh, người dùng, sản phẩm, xem mọi báo cáo tài chính.
2	Staff (Nhân viên quản lý kho/đơn hàng)	Xử lý nhập hàng, chuyển kho, kiểm kho và duyệt/hủy đơn hàng.
3	Cashier (Thu ngân)	Bán hàng tại quầy qua giao diện POS, thanh toán và in hóa đơn.
4	Kênh TMĐT (Webhook Simulator)	Actor bên ngoài (được mô phỏng), đại diện Shopee/TikTok Shop/Lazada gửi đơn hàng vào hệ thống.
5	Hệ thống (Background Job)	Actor tự động: tính giá vốn bình quân, chống bán vượt tồn, hủy đơn quá hạn, gửi cảnh báo tồn thấp.

Bảng 3.2: Danh sách Actor của hệ thống

3.3.2 Use Cases

3.3.2.1 Diagram(s)

Toàn bộ actor và use case liên quan được tổng hợp trong một sơ đồ Use Case duy nhất, thể hiện quan hệ giữa 5 actor và các nhóm chức năng chính của hệ thống.



Hình 3.2: Sơ đồ Use Case tổng quát của hệ thống

Hình 2 — Cần bổ sung hình: *Hình_02_UseCaseDiagram.png* — Use Case Diagram thể hiện các use case chính của Owner/Staff/Cashier và 2 actor hệ thống.

3.3.2.2 Descriptions

ID	Use Case	Actor	Mô tả
UC01	Đăng nhập / Đăng xuất	Owner, Staff, Cashier	Xác thực qua Email/Phone + Password, nhận JWT chứa TenantId/UserId/Role
UC02	Đăng ký Tenant mới	Owner	Tạo không gian dữ liệu cô lập cho cửa hàng mới

ID	Use Case	Actor	Mô tả
UC03	Quản lý chi nhánh/kho	Owner	Tạo, cập nhật, bật/tắt hiển thị chi nhánh và kho hàng
UC04	Quản lý người dùng & phân quyền	Owner	Cấu hình vai trò Owner/Staff/Cashier cho từng tài khoản
UC05	Quản lý danh mục & thương hiệu	Owner	Quản lý Category/Brand phân cấp
UC06	Quản lý sản phẩm & biến thể (SKU)	Owner	Tạo sản phẩm đơn giản/có biến thể, gán SKU duy nhất
UC07	Quản lý mã vạch (Barcode)	Owner	Sinh tự động hoặc nhập tay mã vạch theo SKU
UC08	Thiết lập giá bán	Owner	Cấu hình giá bán lẻ/bán si, tách biệt giá vốn
UC09	Xem sổ cái tồn kho (Ledger)	Owner, Staff	Truy vấn lịch sử biến động tồn kho (chỉ đọc)
UC10	Tạo & xác nhận phiếu nhập hàng	Staff	Ghi nhận nhập hàng, kích hoạt tính lại giá vốn bình quân
UC11	Tạo phiếu chuyển kho	Staff	Chuyển kho 2 bước: xuất tạm (outbound) → xác nhận nhập (inbound)
UC12	Tạo phiên kiểm kho	Staff	Ghi nhận số đếm thực tế, so sánh với hệ thống
UC13	Xử lý bút toán điều chỉnh tồn kho	Staff	Sinh Adjustment Ledger khi có chênh lệch kiểm kho
UC14	Xử lý đơn hàng (duyet/hủy)	Staff	Xem đơn chờ xử lý, duyệt, in phiếu giao hàng hoặc hủy đơn
UC15	Tìm sản phẩm nhanh (Tên/SKU/Barcode)	Cashier	Giao diện POS tối ưu tốc độ tìm kiếm
UC16	Thanh toán tại quầy	Cashier	Thanh toán tiền mặt/QR, in hóa đơn qua Browser Print API
UC17	Checkout nhanh (atomic)	Cashier	Reserve → Confirm → Deduct trong một giao dịch cơ sở dữ liệu
UC18	Xem báo cáo doanh thu & lợi nhuận gộp	Owner	Lọc theo thời gian, chi nhánh, kênh, SKU
UC19	Xem báo cáo tồn kho & tốc độ bán	Owner, Staff	Theo dõi giá trị tồn kho, sản phẩm bán chạy/tồn đọng
UC20	Nhận cảnh báo tồn kho thấp	Owner, Staff	Thông báo dashboard khi available ≤ ngưỡng tối thiểu
UC21	Gửi đơn hàng mô phỏng (Webhook)	Kênh TMĐT	Gửi payload giả lập từ Shopee/TikTok/Lazada để kiểm thử throughput
UC22	Chuẩn hóa đơn hàng đa kênh	Hệ thống	Chuyển đổi đơn hàng từ mọi kênh về Canonical Order Model
UC23	Giữ chỗ tồn kho (Reserve)	Hệ thống	Tăng reserved khi đơn online vào hub, từ chối nếu không đủ available
UC24	Trừ/giải phóng tồn kho (Deduct/Release)	Hệ thống	Trừ on_hand khi Confirmed, giải phóng reserved khi Cancelled
UC25	Tính giá vốn bình quân gia quyền	Hệ thống	Tự động tính lại khi xác nhận phiếu nhập
UC26	Snapshot giá vốn vào đơn hàng	Hệ thống	Chốt AvgCost vào OrderItem.CostPrice khi đơn Confirmed
UC27	Đẩy thông báo real-time	Hệ thống	SignalR đẩy cảnh báo đơn hàng mới đến Admin/POS
UC28	Tự hủy đơn Reserved quá hạn	Hệ thống	Background job (Hangfire) tự động hủy và giải phóng tồn

Bảng 3.3: Danh sách và mô tả Use Case

3.3.3 System Functional Overview

3.3.3.1 Business Process Flow Overview

Vì phạm vi bàn giao của đồ án là thiết kế hệ thống (không lập trình giao diện), luồng nghiệp vụ của OISM được mô tả bằng Sequence Diagram thay vì Screen Flow. Sáu luồng nghiệp vụ trọng yếu được trình bày chi tiết kèm sơ đồ minh họa ở các mục 3.3.3 (Sổ cái & tồn kho), 3.3.5 (Trung tâm đơn hàng) và 3.3.6 (Chống bán vượt tồn) của chương này.

3.3.3.2 Định hướng giao diện (ngoài phạm vi bàn giao chi tiết)

Bảng dưới đây liệt kê các màn hình dự kiến của hệ thống hoàn chỉnh, dùng làm tham chiếu cho việc lập trình Frontend ở giai đoạn tiếp theo. **Các màn hình này chưa được thiết kế mockup chi tiết trong phạm vi đồ án hiện tại.**

#	Feature	Screen (dự kiến)	Mô tả
1	Xác thực	Login	Đăng nhập theo vai trò, điều hướng dashboard tương ứng
2	Sản phẩm	Danh sách/Chi tiết sản phẩm	Quản lý Category, Brand, SKU, Barcode, giá bán
3	Tồn kho	Sổ cái tồn kho (read-only)	Xem lịch sử InventoryTransaction theo SKU/chi nhánh
4	Nhập hàng	Purchase Receipt	Tạo phiếu nhập, xem giá vốn bình quân sau khi xác nhận
5	Chuyển kho	Stock Transfer	Tạo phiếu chuyển, xác nhận outbound/inbound
6	Kiểm kho	Stocktake	Ghi số đếm thực tế, xem chênh lệch
7	Đơn hàng	Order List / Order Detail	Xem, duyệt, hủy đơn theo trạng thái
8	POS	Màn hình bán hàng	Tìm sản phẩm, giỏ hàng, thanh toán nhanh
9	Báo cáo	Revenue Dashboard	Doanh thu, giá vốn, lợi nhuận gộp theo bộ lọc
10	Cảnh báo	Stock Alert Panel	Danh sách SKU dưới ngưỡng tồn tối thiểu

Bảng 3.4: Danh sách màn hình dự kiến (chưa thiết kế mockup chi tiết)

3.3.3.3 Screen Authorization

Chức năng	Owner	Staff	Cashier	Ghi chú
Quản lý Tenant/Chi nhánh/Người dùng	✓	–	–	Chỉ Owner
Quản lý sản phẩm/SKU/giá bán	✓	–	–	Chỉ Owner
Xem sổ cái tồn kho	✓	✓	–	Chỉ đọc
Nhập hàng / Chuyển kho / Kiểm kho	–	✓	–	Staff thực hiện
Xử lý đơn hàng (duyet/hủy)	✓	✓	–	
Bán hàng tại quầy (POS)	–	–	✓	Chỉ Cashier
Báo cáo doanh thu & lợi nhuận gộp	✓	–	–	Dữ liệu tài chính nhạy cảm
Báo cáo tồn kho & cảnh báo	✓	✓	–	

Bảng 3.5: Ma trận phân quyền theo vai trò (RBAC)

3.3.3.4 Non-Screen Functions

#	Feature	System Function	Mô tả
1	Tính giá vốn bình quân gia quyền	Domain Service	Tự động tính lại AvgCost khi xác nhận Purchase Receipt
2	Chống bán vượt tồn kho	Locking Mechanism	Lock hint (WITH (UPDLOCK, ROWLOCK)) khi Reserve/Confirm/Deduct trên StockBalance

#	Feature	System Function	Mô tả
3	Tự hủy đơn Reserved quá hạn	Background Job (Hangfire)	Định kỳ quét đơn Reserved quá thời gian giữ chỗ, tự động Cancel
4	Tổng hợp báo cáo hằng ngày	Background Job (Hangfire)	Tính toán trước dữ liệu báo cáo doanh thu/tồn kho theo lịch
5	Cảnh báo tồn kho thấp	Event Trigger	Kiểm tra available so với ngưỡng, đẩy cảnh báo dashboard
6	Đẩy thông báo real-time	SignalR Hub	Đẩy sự kiện đơn hàng mới tới Admin/POS không cần tải lại trang

Bảng 3.6: Các chức năng không có màn hình (chạy nền/hệ thống)

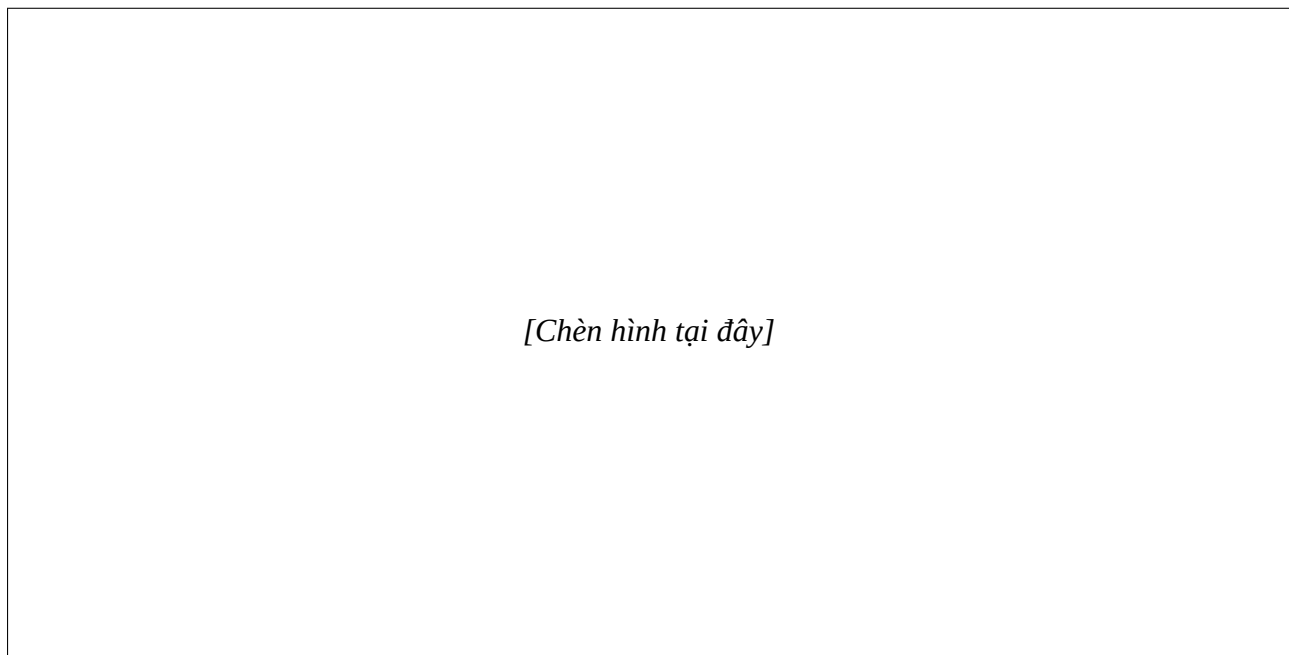
3.3.3.5 Entity Relationship Diagram



Hình 3.3: Sơ đồ ERD toàn bộ cơ sở dữ liệu (20 thực thể)

Hình 3 — Cần bổ sung hình: *Hình_03_ERD_OISM.png* — ERD đầy đủ thể hiện toàn bộ 20 thực thể, quan hệ (1-n, n-n), khóa chính/khóa ngoại. Xem chi tiết danh sách thực thể tại Chương 4, mục 4.2.

3.3.3.6 UML Class Diagram



Hình 3.4: UML Class Diagram các entity chính

Hình 4 — Cần bổ sung hình: *Hình_04_ClassDiagram_OISM.png* — UML Class Diagram các entity chính, kèm thuộc tính, phương thức nghiệp vụ (VD: *Order.Confirm()*, *StockBalance.Reserve()*) và quan hệ multiplicity.

3.3.4 FR-AUTH: Xác thực & Đa khách thuê (Multi-tenant Authentication)

3.3.4.1 Đăng nhập

Actor: Owner, Staff, Cashier

Function trigger: Người dùng nhập Email/Phone + Password

Function description:

- Purpose: Xác thực người dùng và điều hướng theo vai trò.
- Data processing: Xác thực thông tin đăng nhập, tạo JWT Access Token (60 phút) chứa TenantId/UserId/Role, kèm Refresh Token.

Function Details:

- Nếu đăng nhập thành công → điều hướng Dashboard/POS theo role.
- Nếu thất bại → hiển thị thông báo lỗi, không tiết lộ tài khoản có tồn tại hay không (chống dò tài khoản).
- Business Rules: Áp dụng RBAC — mỗi role chỉ thấy chức năng thuộc quyền của mình (xem Bảng 3.5). Mật khẩu băm bằng BCrypt/Argon2 (NFR-SEC-01).

3.3.4.2 Đăng ký Tenant mới

Actor: Owner

Function trigger: Người dùng mới điền form đăng ký cửa hàng

Function description:

- Purpose: Tạo không gian dữ liệu cô lập cho một doanh nghiệp/cửa hàng mới.
- Data processing: Sinh TenantId duy nhất, khởi tạo tài khoản Owner đầu tiên gắn với Tenant đó.

Function Details:

- Business Rules: Mỗi TenantId là ranh giới cô lập dữ liệu tuyệt đối — mọi bảng nghiệp vụ đều áp dụng EF Core Global Query Filter theo TenantId (NFR-TENANT-01).
- Normal Case: Đăng ký hợp lệ → tạo Tenant + tài khoản Owner → chuyển hướng đăng nhập.
- Abnormal Case: Email đã tồn tại ở một Tenant khác → cho phép (vì tách biệt theo Tenant), nhưng trùng trong cùng Tenant → báo lỗi.

3.3.4.3 Các chức năng còn lại của nhóm FR-AUTH

Mã	Tên chức năng	Tóm tắt
FR-AUTH-03	Role-Based Access Control	Cấu hình và kiểm soát quyền API/màn hình theo 3 vai trò
FR-AUTH-04	Branch Management	Owner tạo, cập nhật, bật/tắt hiển thị chi nhánh và kho hàng thuộc quyền sở hữu

Bảng 3.7: Tóm tắt các FR còn lại — nhóm FR-AUTH

3.3.5 FR-PROD: Quản lý sản phẩm & biến thể (Product & Variant Management)

3.3.5.1 Quản lý sản phẩm & biến thể/SKU

Actor: Owner

Function trigger: Owner chọn "Thêm sản phẩm mới"

Function description:

- Purpose: Hỗ trợ sản phẩm đơn giản và sản phẩm có biến thể (màu, size...).
- Data processing: Tạo bản ghi Product, sinh các ProductVariant (SKU) tương ứng với từng tổ hợp thuộc tính.

Function Details:

- Validation: SKUCode phải duy nhất trong phạm vi một TenantId.
- Business Rules: Mỗi biến thể là đơn vị quản lý tồn kho độc lập (không quản lý tồn theo Product cha).
- Normal Case: Nhập đủ thông tin → lưu Product + Variant thành công.
- Abnormal Case: SKUCode trùng trong cùng Tenant → báo lỗi validation.

3.3.5.2 Các chức năng còn lại của nhóm FR-PROD

Mã	Tên chức năng	Tóm tắt
FR-PROD-01	Category & Brand Management	Quản lý danh mục sản phẩm phân cấp và thương hiệu
FR-PROD-03	Barcode Management	Tự sinh/nhập tay mã vạch EAN-13/Code128 theo SKU

Mã	Tên chức năng	Tóm tắt
FR-PROD-04	Listed Price Management	Thiết lập giá bán lẻ/bán sỉ theo SKU, tách biệt giá vốn

Bảng 3.8: Tóm tắt các FR còn lại — nhóm FR-PROD

3.3.6 FR-INV: Sổ cái & Quản lý tồn kho (Inventory Ledger & Stock Management)

3.3.6.1 Ghi nhận sổ cái tồn kho (Inventory Ledger Recording)

Actor: Hệ thống (kích hoạt bởi mọi nghiệp vụ liên quan tồn kho)
Function trigger: Bất kỳ nghiệp vụ nào làm thay đổi tồn kho (nhập, xuất, bán, trả, kiểm kho, chuyển kho)
Function description:

- Purpose: Ghi nhận tuyệt đối mọi biến động tồn kho, đảm bảo khả năng truy vết/kiểm toán.
- Data processing: Chèn (INSERT) một dòng mới vào bảng InventoryTransaction: TenantId, BranchId, SKUId, Type (IN/OUT), Quantity, BalanceAfter, ReferenceId, CreatedAt.

Function Details:

- Business Rules (**quan trọng nhất của toàn hệ thống**): Bảng InventoryTransaction là append-only tuyệt đối — cấm UPDATE/DELETE ở mọi tầng, kể cả truy cập SQL trực tiếp (NFR-SEC-03). Cơ chế thực thi bằng trigger/rule ở tầng SQL Server (chi tiết tại Chương 4, mục 4.2).
- Mọi điều chỉnh sai sót phải tạo dòng bù trừ mới (compensating transaction), không được sửa dòng gốc.

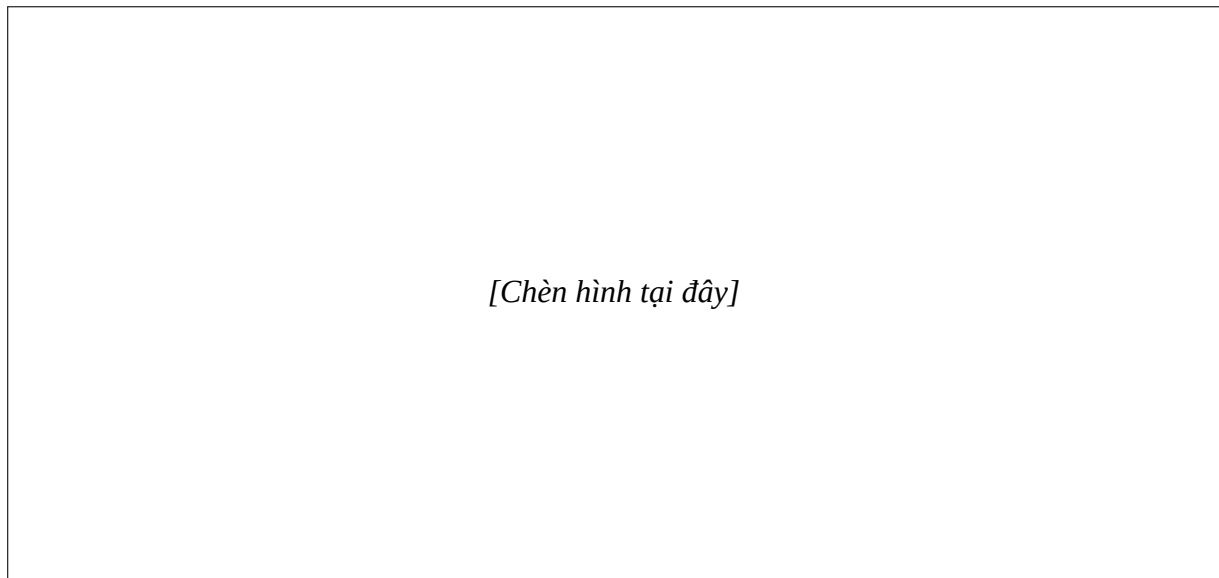
3.3.6.2 Nhập hàng & tính giá vốn bình quân (Purchase Receipt Management)

Actor: Staff
Function trigger: Staff tạo phiếu nhập hàng và xác nhận
Function description:

- Purpose: Ghi nhận nhập hàng từ nhà cung cấp, tự động cập nhật giá vốn.
- Data processing: Với mỗi dòng SKU trong phiếu, tính lại giá vốn theo công thức bình quân gia quyền, ghi Ledger loại IN, cập nhật StockBalance.OnHand và AvgCost.

Function Details:

- Business Rules: $\text{New Cost} = ((\text{Old Stock} \times \text{Old Cost}) + (\text{Import Qty} \times \text{Import Price})) / (\text{Old Stock} + \text{Import Qty})$.
- Normal Case: Xác nhận phiếu → tính giá vốn mới → ghi Ledger thành công trong 1 transaction.
- Abnormal Case: Phiếu có dòng SKU không tồn tại trong Tenant → từ chối toàn bộ phiếu (rollback).



Hình 3.5: Sequence Diagram — luồng nhập hàng & tính giá vốn bình quân

Hình 7 — Cần bổ sung hình: *Hình_07_Sequence_PurchaseReceipt.png* — *PurchaseReceipt* → tính *AvgCost* → cập nhật *StockBalance* → ghi *Ledger IN*.

3.3.6.3 Chuyển kho (Stock Transfer Management)

Actor: Staff

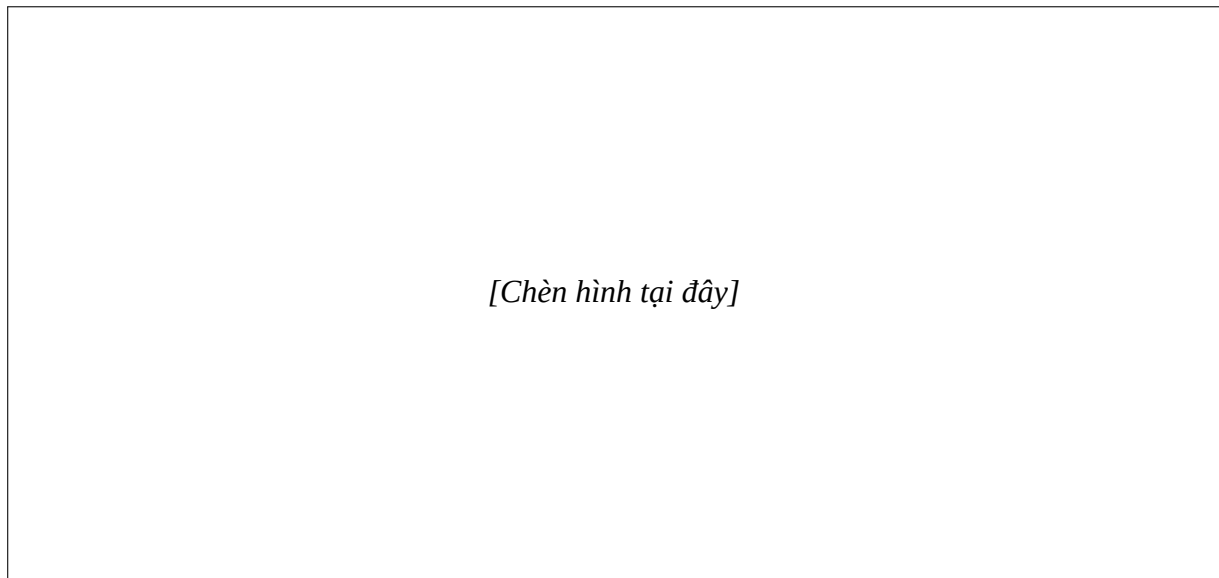
Function trigger: Staff tạo phiếu chuyển kho giữa 2 chi nhánh

Function description:

- Purpose: Di chuyển tồn kho giữa các chi nhánh mà vẫn giữ tính toàn vẹn Ledger.
- Data processing: Quy trình 2 bước — Outbound (xuất tạm tại chi nhánh nguồn) và Inbound (xác nhận nhập tại chi nhánh đích).

Function Details:

- Normal Case: Outbound ghi Ledger OUT tại nguồn (trạng thái InTransit) → Inbound ghi Ledger IN tại đích (trạng thái Completed).
- Abnormal Case: Chi nhánh đích từ chối nhận hàng → cần quy trình hoàn trả riêng (ghi Ledger IN ngược lại tại chi nhánh nguồn).



Hình 3.6: Sequence Diagram — luồng chuyển kho 2 bước (Outbound/Inbound)

Hình 8 — Cần bổ sung hình: *Hình_08_Sequence_StockTransfer.png* — 2 bước Outbound/Inbound giữa 2 chi nhánh.

3.3.6.4 Kiểm kho (Stocktake Management)

Actor: Staff

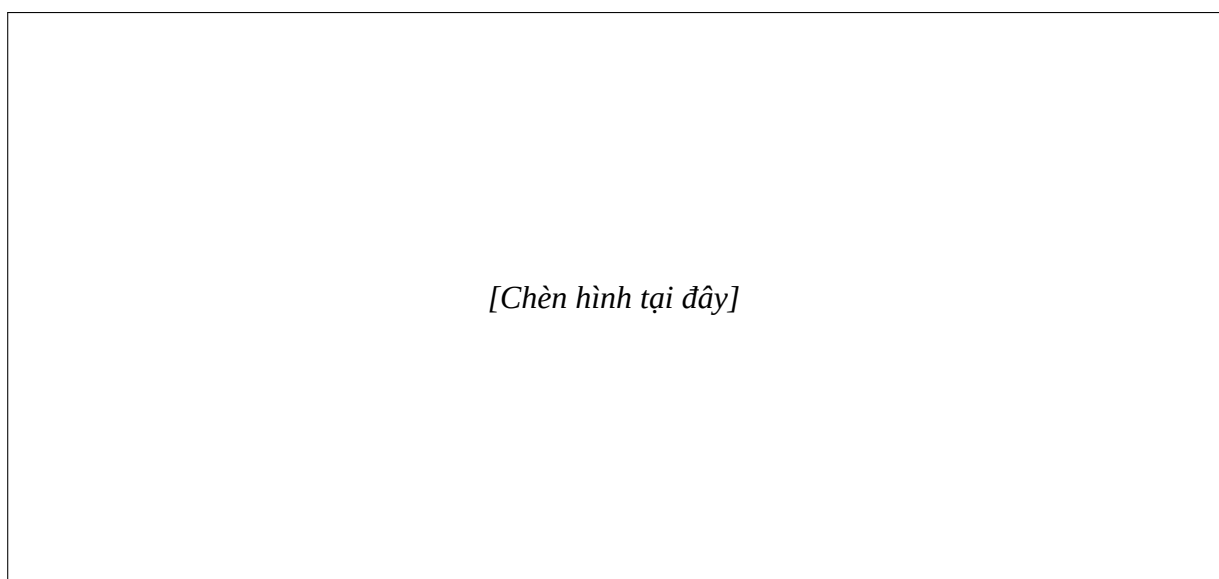
Function trigger: Staff tạo phiên kiểm kho cho một chi nhánh

Function description:

- Purpose: Đối chiếu tồn kho thực tế với số liệu hệ thống.
- Data processing: Ghi CountedQty theo từng SKU, so sánh với SystemQty (từ StockBalance) để tính VarianceQty.

Function Details:

- Business Rules: Nếu có chênh lệch, sinh bút toán điều chỉnh (Adjustment) vào Ledger để đưa OnHand khớp với số đếm thực tế — không sửa trực tiếp OnHand.



Hình 3.7: Sequence Diagram — luồng kiểm kho & sinh bút toán điều chỉnh

Hình 9 — Cần bổ sung hình: *Hình_09_Sequence_Stocktake.png* — tạo phiên kiểm kho → so sánh → sinh bút toán Adjustment.

3.3.7 FR-COST: Tính giá vốn bình quân gia quyền (Weighted Average Cost Calculation)

3.3.7.1 Tự động tính giá vốn (Automated Batch COGS Calculation)

Actor: Hệ thống (kích hoạt khi xác nhận Purchase Receipt)

Function trigger: Staff xác nhận phiếu nhập hàng (xem mục 3.3.3.2)

Function description:

- Purpose: Đảm bảo giá vốn luôn phản ánh đúng chi phí thực tế khi nhập hàng nhiều đợt với giá khác nhau.
- Data processing: Cập nhật StockBalance.AvgCost theo công thức bình quân gia quyền ngay khi phiếu nhập được xác nhận, trong cùng transaction với việc ghi Ledger.

Function Details:

- Business Rules: Đây là phép tính **bắt buộc đồng bộ** (không cho phép xử lý bất đồng bộ/độ trễ) vì ảnh hưởng trực tiếp đến các đơn hàng được Confirm ngay sau đó.

3.3.7.2 Snapshot giá vốn khi xuất kho (COGS Snapshot on Dispatch)

Actor: Hệ thống

Function trigger: Đơn hàng chuyển sang trạng thái Confirmed hoặc thanh toán tại POS

Function description:

- Purpose: Đảm bảo báo cáo lợi nhuận không bị thay đổi ngược thời gian khi giá vốn trung bình biến động sau đó.
- Data processing: Chốt (snapshot) giá trị AvgCost hiện tại của SKU vào OrderItem.CostPrice tại thời điểm Confirm.

Function Details:

- Business Rules: OrderItem.CostPrice là bất biến sau khi ghi — không cập nhật lại kể cả khi AvgCost của SKU thay đổi sau đó (đảm bảo tính đúng đắn dẫn lịch sử báo cáo lợi nhuận theo từng thời điểm bán).

3.3.8 FR-ORD: Trung tâm đơn hàng tập trung (Centralized Order Hub)

3.3.8.1 Vòng đời đơn hàng (Order State Machine)

Actor: Hệ thống, Staff (thao tác chuyển trạng thái thủ công khi cần)

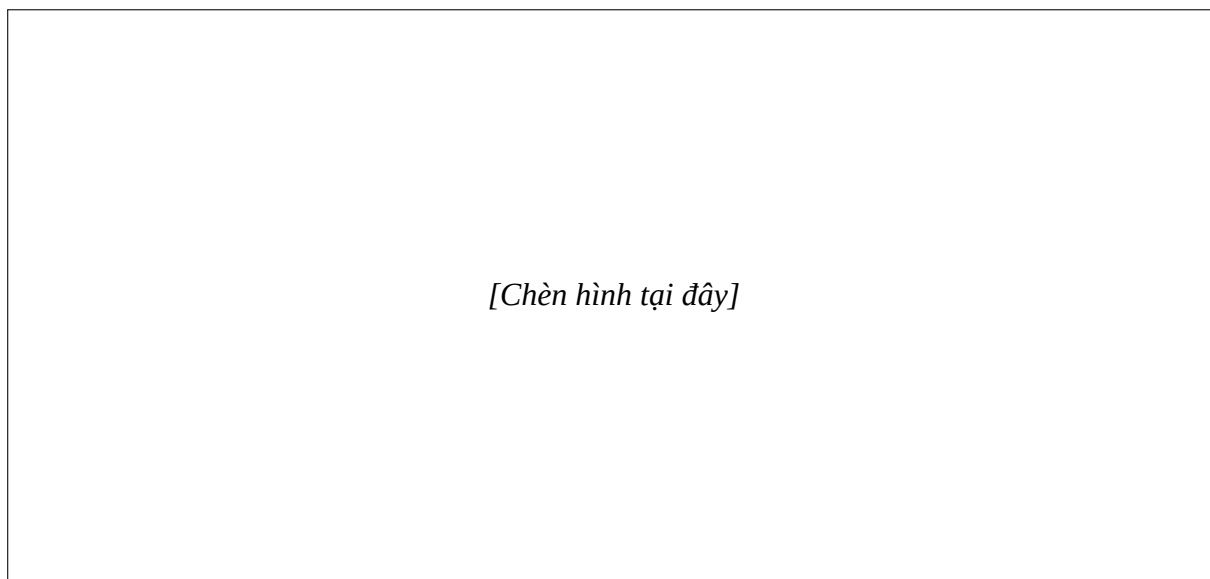
Function trigger: Đơn hàng được tạo từ bất kỳ kênh nào (POS, Admin, Webhook)

Function description:

- Purpose: Đảm bảo mọi đơn hàng tuân theo đúng một vòng đời nghiêm ngặt, không có trạng thái mơ hồ.
- Data processing: Order.Status chuyển tuần tự Draft → Reserved → Confirmed → Completed, với nhánh rẽ Cancelled có thể xảy ra từ Draft, Reserved hoặc Confirmed.

Function Details:

- Business Rules: Mọi chuyển trạng thái không hợp lệ (ví dụ Completed → Reserved) phải bị từ chối ngay ở tầng Domain, không phụ thuộc vào việc kiểm tra ở tầng giao diện.



Hình 3.8: State Diagram — vòng đời đơn hàng (Order State Machine)

Hình 10 — Cần bổ sung hình: *Hình_10_StateDiagram_OrderLifecycle.png* — toàn bộ trạng thái và điều kiện chuyển trạng thái hợp lệ của Order.

3.3.8.2 Chuẩn hóa đơn hàng đa kênh & Xử lý đơn

Actor: Kênh TMĐT (Webhook), Staff

Function trigger: Webhook Simulator gửi payload / Staff tạo đơn thủ công tại Admin

Function description:

- Purpose: Chuẩn hóa đơn hàng từ mọi kênh về một Canonical Order Model duy nhất.
- Data processing: Ánh xạ payload đầu vào (khác định dạng theo từng kênh) về cùng một schema Order/OrderItem nội bộ.

Function Details:

- Normal Case: Staff xem đơn chờ xử lý, duyệt, in phiếu giao hàng hoặc hủy đơn (FR-ORD-03).
- Abnormal Case: Payload webhook thiếu trường bắt buộc → từ chối, ghi log lỗi tích hợp.

3.3.9 FR-RSE: Cơ chế chống bán vượt tồn (Anti-Oversell Mechanism)

3.3.9.1 Giữ chỗ & Trừ/Giải phóng tồn kho (Reserve / Deduct / Release)

Actor: Hệ thống

Function trigger: Đơn hàng online vào Order Hub (Reserve); đơn chuyển Confirmed (Deduct) hoặc Cancelled (Release)

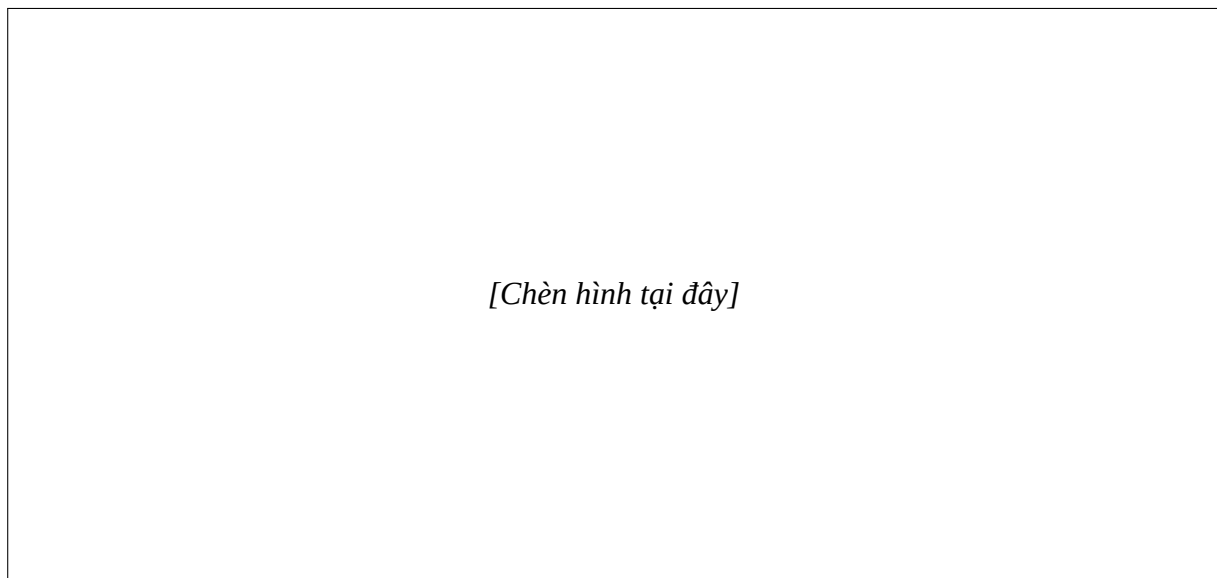
Function description:

- Purpose: Đây là chức năng **quan trọng nhất** của toàn hệ thống — loại bỏ tuyệt đối khả năng bán vượt tồn kho ngay cả khi nhiều đơn hàng tranh chấp đồng thời.

- Data processing: Tính available = on_hand – reserved theo thời gian thực; khóa dòng StockBalance bằng lock hint (WITH (UPDLOCK, ROWLOCK)) trước khi tăng/giảm reserved hoặc on_hand.

Function Details:

- Validation: Nếu available < số lượng đặt mua → từ chối đơn (reject), trả lỗi 409 Conflict.
- Business Rules (**NFR-PERF-02**): Bắt buộc dùng Pessimistic Locking (lock hint WITH (UPDLOCK, ROWLOCK)) hoặc Optimistic Locking (Version Check) ở tầng cơ sở dữ liệu — không được xử lý kiểm tra tồn kho chỉ ở tầng ứng dụng, vì điều này không đủ để chống race-condition khi có nhiều request đồng thời.
- Normal Case: Reserve thành công → tăng reserved, đơn chuyển Reserved. Confirm → trừ on_hand, giảm reserved, ghi Ledger OUT. Cancel → giảm reserved, trả lại tồn khả dụng.
- Abnormal Case (kịch bản Flash Sale): 50 đơn hàng đồng thời tranh 1 SKU chỉ còn 1 đơn vị tồn → chỉ đúng 1 đơn được Reserve thành công, 49 đơn còn lại bị reject — không có trường hợp available âm.



Hình 3.9: Sequence Diagram — luồng giữ chỗ tồn kho (Stock Reservation)

Hình 5 — Cần bổ sung hình: *Hình_05_Sequence_StockReservation.png* — Order Hub → StockBalance (lock) → Ledger, thể hiện nhánh reject khi thiếu tồn.

3.3.10 FR-POS: Bán hàng tại quầy — PWA POS (Counter Sales)

3.3.10.1 Checkout nhanh tại quầy (Fast Checkout Workflow)

Actor: Cashier

Function trigger: Cashier nhấn "Thanh toán" sau khi thêm sản phẩm vào giỏ hàng

Function description:

- Purpose: Hoàn tất một giao dịch bán hàng tại quầy trong thời gian ngắn nhất, vẫn đảm bảo không bán vượt tồn khả dụng.
- Data processing: Thực hiện chuỗi Reserve → Confirm → Deduct trong **một giao dịch cơ sở dữ liệu nguyên tử (atomic transaction)** duy nhất.

Function Details:

- Validation: Kiểm tra available tức thời khi Cashier chọn sản phẩm (FR-POS-02), chặn thêm vào giỏ nếu vượt tồn khả dụng — bảo vệ phần tồn đã được giữ chỗ bởi đơn online.
- Business Rules (NFR-SEC-02): Nếu bất kỳ bước nào trong chuỗi lỗi (ví dụ hết tồn giữa chừng do đơn khác vừa xử lý xong) → rollback toàn bộ transaction, không lưu trạng thái trung gian.
- Normal Case: Thanh toán thành công → in hóa đơn qua Browser Print API, ghi Ledger OUT, snapshot CostPrice.



Hình 3.10: Sequence Diagram — luồng thanh toán nhanh tại POS (Fast Checkout)

Hình 6 — Cần bổ sung hình: *Hình_06_Sequence_FastCheckout.png* — POS → Order → StockBalance → Ledger trong 1 transaction, kèm nhánh rollback.

3.3.10.2 Các chức năng còn lại của nhóm FR-POS

Mã	Tên chức năng	Tóm tắt
FR-POS-01	Fast Sales Interface	Giao diện tối ưu cảm ứng/phím tắt, tìm sản phẩm theo Tên/SKU/Barcode
FR-POS-03	Checkout & Receipt Printing	Thanh toán tiền mặt/QR, in hóa đơn qua Browser Print API

Bảng 3.9: Tóm tắt các FR còn lại — nhóm FR-POS

3.3.11 FR-REP: Báo cáo & Cảnh báo (Reports & Alerts)

Mã	Tên chức năng	Tóm tắt
FR-REP-01	Revenue & Gross Profit Report	Doanh thu thuần, tổng giá vốn, lợi nhuận gộp; lọc theo thời gian/chi nhánh/kênh/SKU
FR-REP-02	Inventory & Velocity Report	Giá trị tồn kho theo giá vốn, sản phẩm bán chạy/tồn đọng chậm

Mã	Tên chức năng	Tóm tắt
FR-REP-03	Stock Alert	Thông báo tự động khi available $\leq$ ngưỡng đặt hàng lại tối thiểu theo SKU

Bảng 3.10: Tóm tắt chức năng — nhóm FR-REP

3.3.12 FR-SIM: Webhook Simulator & Xử lý thời gian thực

Mã	Tên chức năng	Tóm tắt
FR-SIM-01	E-commerce Webhook Simulator	Gửi payload giả lập Shopee/TikTok/Lazada để kiểm tra throughput & locking
FR-SIM-02	Real-time Push Notification	SignalR đẩy cảnh báo đơn hàng mới đến Admin/POS
FR-SIM-03	Background Processing	Hangfire: tự hủy đơn Reserved quá hạn, tổng hợp báo cáo hằng ngày

Bảng 3.11: Tóm tắt chức năng — nhóm FR-SIM

3.3.13 External Interfaces

- **Giao diện người dùng:** Admin Dashboard (desktop, tối ưu cho thao tác quản trị) và POS App dạng PWA (tối ưu cho tablet/thiết bị cầm tay, responsive).
- **Tích hợp bên ngoài:** Webhook Simulator mô phỏng payload từ Shopee/TikTok Shop/Lazada; kiến trúc cho phép thay thế bằng API thật của các sàn ở giai đoạn phát triển tiếp theo.
- **Thiết bị phần cứng:** Máy quét mã vạch (barcode scanner) qua giao diện bàn phím HID chuẩn tại POS; máy in hóa đơn qua Browser Print API.

3.3.14 Quality Attributes

3.3.14.1 Performance

- API Response Time: tìm SKU/quét mã vạch tại POS p95 < 200ms; giao dịch tạo đơn/ghi < 500ms; báo cáo doanh thu tổng hợp < 2s với tập dữ liệu dưới 100.000 bản ghi.
- Concurrency & Anti-Oversell: xử lý được kịch bản Flash Sale (50 đơn online đồng thời tranh 1 SKU còn 1 đơn vị tồn), bắt buộc dùng Pessimistic Locking hoặc Optimistic Locking để tuyệt đối không cho tồn kho khả dụng âm.

3.3.14.2 Reliability & Data Integrity

- Tuân thủ ACID: chuỗi Order Creation → Reserve → Deduct bắt buộc thực thi trong một giao dịch cơ sở dữ liệu nguyên tử, rollback toàn bộ nếu thất bại.
- Tính kiểm toán của Ledger: bảng InventoryLedger là append-only tuyệt đối, không cho phép UPDATE/DELETE; mọi điều chỉnh phải qua bút toán bù trừ.
- Mục tiêu uptime 99.5% trong giờ hoạt động (7:00–22:00) cho hệ thống hoàn chỉnh.

3.3.14.3 Security

- Mật khẩu người dùng băm bằng BCrypt/Argon2. Giao tiếp API qua HTTPS/TLS 1.3.
- JWT Access Token thời hạn ngắn (60 phút) kèm Refresh Token an toàn.

3.3.14.4 Multi-tenancy & Scalability

- Mô hình Shared Database, Separate Schema/TenantId. Mọi bảng nghiệp vụ đều có TenantId, bắt buộc dùng EF Core Global Query Filters để đảm bảo không rò rỉ dữ liệu giữa các tenant.
- Tối ưu index trên (TenantId, CreatedAt) và (TenantId, SKUId) cho bảng InventoryLedger và Orders để giữ hiệu năng truy vấn ở quy mô hàng triệu dòng ledger.

3.3.14.5 Usability

- Giao diện POS tối giản, thanh toán dưới 3 lần click hoặc quét mã vạch + Enter.
- Responsive cho desktop, tablet, thiết bị POS cầm tay.

3.3.14.6 Maintainability

- Tách biệt nghiêm ngặt theo Clean Architecture: Domain (COGS, Reserve), Application (use case), Infrastructure (EF Core, SQL Server, SignalR), Presentation (Web API).
- Các module lõi (Inventory Ledger, Anti-Oversell Reserve, COGS) phải đạt tối thiểu 80% Unit & Integration Test coverage khi được lập trình ở giai đoạn tiếp theo.

3.3.15 Business Rules

ID	Rule Definition
BR-01	Bảng InventoryTransaction là append-only tuyệt đối — không có thao tác UPDATE/DELETE ở bất kỳ tầng nào, kể cả truy cập SQL trực tiếp.
BR-02	available = on_hand – reserved không được âm tại bất kỳ thời điểm nào.
BR-03	SKUCode là duy nhất trong phạm vi một TenantId (unique composite key: TenantId + SKUCode).
BR-04	OrderItem.CostPrice là snapshot bất biến sau khi đơn chuyển sang Confirmed, không cập nhật lại kể cả khi AvgCost của SKU thay đổi sau đó.
BR-05	Mọi bảng nghiệp vụ phải có cột TenantId và áp dụng Global Query Filter để đảm bảo cô lập dữ liệu đa khách thuê.
BR-06	Chuỗi giao dịch Reserve → Confirm → Deduct phải thực thi trong một transaction nguyên tử duy nhất, rollback toàn bộ nếu bất kỳ bước nào thất bại.
BR-07	Mọi điều chỉnh sai sót tồn kho phải tạo dòng bù trừ mới (compensating transaction) vào Ledger, không được sửa dòng gốc.
BR-08	Giờ rời bến/chuyển trạng thái đơn hàng không hợp lệ (VD: Completed → Reserved) phải bị từ chối ở tầng Domain.
BR-09	Cơ chế chống bán vượt tồn bắt buộc dùng khóa cấp cơ sở dữ liệu (Pessimistic hoặc Optimistic Locking), không được xử lý chỉ ở tầng ứng dụng.
BR-10	Chỉ Owner mới có quyền xem báo cáo doanh thu & lợi nhuận gộp (dữ liệu tài chính nhạy cảm).

Bảng 3.12: Danh sách quy tắc nghiệp vụ (Business Rules)

3.3.16 Common Requirements

- **Authentication and Authorization:** Áp dụng RBAC (Role-Based Access Control) để đảm bảo mỗi vai trò (Owner/Staff/Cashier) chỉ truy cập chức năng phù hợp.
- **Data Integrity:** Mọi biến động tồn kho phải đi qua sổ cái append-only, đảm bảo có thể truy vết và kiểm toán 100% lịch sử thay đổi.
- **Multi-tenancy:** Hệ thống phải mở rộng được để phục vụ nhiều cửa hàng đồng thời mà không rò rỉ dữ liệu chéo giữa các Tenant.
- **Concurrency Safety:** Các module liên quan tới tồn kho (Reserve/Confirm/Deduct) phải an toàn khi có nhiều request đồng thời, không phụ thuộc vào kiểm tra ở tầng giao diện.

3.3.17 Application Messages List

#	Mã	Loại	Ngữ cảnh	Nội dung
1	MSG01	Error	Giữ chỗ tồn kho	"Không đủ tồn kho khả dụng cho sản phẩm này."(HTTP 409 Conflict)
2	MSG02	Success	Giữ chỗ tồn kho	"Giữ chỗ tồn kho thành công."
3	MSG03	Error	Checkout POS	"Giao dịch thất bại: tồn kho đã thay đổi, vui lòng thử lại."
4	MSG04	Success	Checkout POS	"Thanh toán thành công."
5	MSG05	Error	Nhập hàng	"Phiếu nhập chứa SKU không tồn tại trong hệ thống."
6	MSG06	Success	Nhập hàng	"Xác nhận phiếu nhập thành công, đã cập nhật giá vốn bình quân."
7	MSG07	Warning	Báo cáo tồn kho	"SKU [x] đã chạm ngưỡng tồn kho tối thiểu."
8	MSG08	Error	Sổ cái tồn kho	"Thao tác không được phép: InventoryTransaction chỉ cho phép ghi thêm (append-only)."
9	MSG09	Info	Đơn hàng	"Đơn hàng đã tự động bị hủy do quá thời gian giữ chỗ."
10	MSG10	Error	Đăng nhập	"Email/mật khẩu không chính xác."

Bảng 3.13: Danh sách thông báo ứng dụng

3.3.18 Bảng tóm tắt yêu cầu theo nhóm chức năng

Nhóm chức năng	Vai trò chính	Ưu tiên	Mô tả giá trị
FR-AUTH: Xác thực & đa khách thuê	Owner	Cao	Cô lập dữ liệu tuyệt đối giữa các Tenant, phân quyền theo vai trò
FR-PROD: Quản lý sản phẩm/biến thể	Owner	Cao	Chuẩn hóa danh mục, SKU, barcode, giá bán
FR-INV: Sổ cái & tồn kho	Staff	Rất cao	Ghi nhận mọi biến động tồn kho, không thể sửa/xóa
FR-COST: Giá vốn bình quân	Hệ thống (tự động)	Rất cao	Tính đúng lợi nhuận gộp theo thời gian thực
FR-ORD: Trung tâm đơn hàng	Staff	Rất cao	Chuẩn hóa đơn từ mọi kênh, vòng đời rõ ràng

Nhóm chức năng	Vai trò chính	Ưu tiên	Mô tả giá trị
FR-RSE: Chống bán vượt tồn	Hệ thống (tự động)	Rất cao	Loại bỏ hoàn toàn tình trạng oversell
FR-POS: Bán hàng tại quầy	Cashier	Cao	Thanh toán nhanh, an toàn, atomic
FR-REP: Báo cáo & cảnh báo	Owner	Cao	Ra quyết định kinh doanh dựa trên số liệu chính xác
FR-SIM: Webhook & real-time	Hệ thống (tự động)	Trung bình	Mô phỏng đa kênh, thông báo tức thời

Bảng 3.14: Bảng tóm tắt yêu cầu theo chức năng chính

3.3.19 Kết luận đặc tả yêu cầu

Các yêu cầu trong Chương 3 tập trung mô tả đầy đủ bối cảnh sử dụng, người dùng, chức năng, dữ liệu, giao diện và các ràng buộc kỹ thuật của OISM. Điều này cho phép các bên liên quan thống nhất quan điểm về sản phẩm, đồng thời tạo nền tảng vững chắc để triển khai cơ sở dữ liệu (Chương 4) theo hướng có kiểm soát, có thể kiểm chứng và dễ mở rộng trong tương lai.

Quy tắc xác thực Mỗi vai trò (Owner/Staff/Cashier) tương ứng với một nhóm quyền truy cập riêng, thực thi qua JWT chứa TenantId/UserId/Role và middleware RBAC. Điều này giảm rủi ro truy cập trái phép và đảm bảo ranh giới trách nhiệm rõ ràng giữa các vai trò.

Quy tắc dữ liệu Mọi biến động tồn kho phải được ghi vào sổ cái InventoryTransaction dạng append-only, có TenantId, BranchId, SKUId, loại giao dịch, số lượng, số dư sau giao dịch và thời điểm tạo — không được sửa hoặc xóa sau khi ghi.

Quy tắc tích hợp Hệ thống cần khả năng tiếp nhận đơn hàng từ nhiều kênh (POS, Admin, Webhook Simulator) thông qua một mô hình đơn hàng chuẩn hóa duy nhất (Canonical Order Model), với xác thực và xử lý lỗi rõ ràng ở mọi endpoint.

Quy tắc hoạt động Hệ thống phải hoạt động ổn định ngay cả khi nhiều đơn hàng tranh chấp đồng thời trên cùng một SKU (kịch bản flash sale). Mọi logic giữ chỗ/trừ tồn phải có kiểm tra ràng buộc chặt chẽ ở tầng cơ sở dữ liệu (locking), không chỉ ở tầng ứng dụng.

Đánh giá mức độ hoàn thiện Khi yêu cầu ở Chương 3 được xác nhận, nhóm có đủ cơ sở để triển khai thiết kế dữ liệu chi tiết và cơ sở dữ liệu vật lý ở Chương 4. Các thành phần về nghiệp vụ, phân quyền và ràng buộc dữ liệu đã được nêu rõ để hỗ trợ việc viết script DDL và seed data có hướng dẫn cụ thể.

3.4 Tổng hợp yêu cầu chức năng và phi chức năng

Bảng dưới đây tổng hợp các nhóm yêu cầu chính để phục vụ quản lý dự án và triển khai cơ sở dữ liệu theo từng giai đoạn.

Nhóm yêu cầu	Mức ưu tiên	Đơn vị phụ trách	Kết quả mong đợi
Sổ cái tồn kho (Ledger)	Rất cao	TV1 (Data Architect)	Bảng append-only, có trigger chặn UPDATE/DELETE
Cơ chế chống bán vượt tồn	Rất cao	TV1, TV3	Thiết kế locking đúng, không có available âm

Nhóm yêu cầu	Mức ưu tiên	Đơn vị phụ trách	Kết quả mong đợi
Tính giá vốn bình quân	Rất cao	TV1, TV2	Công thức đúng, kiểm chứng bằng seed data mẫu
Đa khách thuê (Multi-tenant)	Cao	TV1	Mọi bảng có TenantId, cô lập dữ liệu
Đặc tả API	Cao	TV3	Swagger/Postman đầy đủ endpoint
Báo cáo & cảnh báo	Trung bình	TV1, TV2	Schema hỗ trợ truy vấn báo cáo hiệu quả

Bảng 3.15: Bảng tổng hợp yêu cầu chức năng và phi chức năng theo ưu tiên

3.5 Mức độ sẵn sàng cho giai đoạn thiết kế cơ sở dữ liệu

Một đặc tả yêu cầu tốt phải cho phép chuyển tiếp sang thiết kế dữ liệu chi tiết mà không cần giải thích thêm nhiều. Đối với OISM, các yêu cầu cốt lõi đã được phân tách theo thực thể dữ liệu, quy trình nghiệp vụ, phân quyền và ràng buộc toàn vẹn — cho phép triển khai schema SQL Server với sự tách biệt rõ ràng giữa các nhóm bảng (Tenant/Auth, Catalog, Inventory Ledger, Order, Reporting).

- Đảm bảo mọi bảng nghiệp vụ có TenantId và Global Query Filter tương ứng.
- Đảm bảo số cái tồn kho append-only tuyệt đối, có cơ chế chặn ở tầng database.
- Hỗ trợ khóa hàng (row-level locking) để chống race-condition trong kịch bản flash sale.
- Tối ưu index cho truy vấn báo cáo ở quy mô hàng triệu dòng ledger.
- Đảm bảo tính atomic cho chuỗi giao dịch Reserve → Confirm → Deduct.

3.6 Nhận định cuối cùng

Chương 3 đã xác định rõ: hệ thống cần phục vụ nhiều vai trò, nhiều kênh bán và phải đảm bảo tính đúng đắn tuyệt đối của dữ liệu tồn kho trong điều kiện truy cập đồng thời cao. Dựa trên các yêu cầu này, cơ sở dữ liệu có thể được thiết kế theo hướng chặt chẽ, có thể kiểm chứng và sẵn sàng làm nền tảng cho việc lập trình Backend ở các học kỳ tiếp theo. Đây là phần nền tảng để triển khai và kiểm thử cơ sở dữ liệu ở Chương 4.

Chương 4

Triển khai Cơ sở dữ liệu và Kiểm thử

Mục tiêu của Chương 4: Khác với một đồ án lập trình đầy đủ, phạm vi bàn giao thực tế của nhóm không bao gồm Backend/Frontend (mục 1.6). Vì vậy, "hiện thực" trong chương này được hiểu là **triển khai cơ sở dữ liệu vật lý** từ thiết kế ở Chương 3 — bao gồm viết script DDL, cơ chế ràng buộc append-only, seed data, và kiểm thử ở mức dữ liệu.

4.1 Môi trường và công nghệ sử dụng

4.1.1 Công nghệ đã sử dụng trong phạm vi bàn giao (Thiết kế + Database)

- **Cơ sở dữ liệu:** Microsoft SQL Server 2022, chạy qua Docker (image `mcr.microsoft.com/mssql/server`) để đảm bảo tính nhất quán môi trường giữa các thành viên.
- **Công cụ thiết kế dữ liệu:** dbdiagram.io (ERD), draw.io (Class Diagram, Sequence Diagram, State Diagram).
- **Công cụ quản lý CSDL:** SQL Server Management Studio (SSMS) / Azure Data Studio để chạy thử script DDL, seed data và script kiểm tra toàn vẹn.
- **Đặc tả API:** Swagger Editor / Postman (dùng để đặc tả, không cần Backend thật để chạy).
- **Quản lý mã nguồn:** GitHub (script SQL, tài liệu thiết kế).

4.1.2 Công nghệ đã hoạch định cho hệ thống hoàn chỉnh (chưa triển khai trong học kỳ này)

Các công nghệ dưới đây đã được xác định trong thiết kế kiến trúc (Chương 3) làm cơ sở cho việc lập trình Backend/Frontend ở giai đoạn tiếp theo:

- **Backend:** .NET 8 Web API, theo Clean Architecture (Domain/Application/Infrastructure/Presentation), truy cập dữ liệu qua Entity Framework Core với provider `Microsoft.EntityFrameworkCore.SqlServer`.
- **Frontend:** ReactJS cho Admin Dashboard; ReactJS PWA cho POS Application.
- **Giao tiếp real-time:** SignalR (WebSocket).
- **Background Job:** Hangfire (hỗ trợ sẵn storage provider cho SQL Server).
- **Containerization & CI/CD:** Docker, Docker Compose, GitHub Actions.

4.1.3 Môi trường phát triển

- **Hệ quản trị CSDL:** SQL Server 2022 container, mapping port 1433, volume lưu trữ dữ liệu bền vững giữa các lần chạy.
- **Quản lý mã nguồn:** Git/GitHub, quy ước nhánh main, feature/erd, feature/ddl-schema, feature/seed-data.
- **Môi trường:** Một môi trường development duy nhất trong phạm vi đồ án (chưa phân tách staging/production vì chưa triển khai thực tế).

Lưu ý chuyển đổi hệ quản trị CSDL: Thiết kế logic (ERD, chuẩn hóa, Business Rules ở Chương 3) hoàn toàn độc lập với hệ quản trị CSDL. Nhóm chọn SQL Server thay vì PostgreSQL cho bản triển khai vật lý vì phù hợp hơn với hệ sinh thái .NET/EF Core đã hoạch định cho Backend, và vì mức độ phổ biến của SQL Server trong môi trường doanh nghiệp Việt Nam.

4.2 Thiết kế Logic và Vật lý Cơ sở dữ liệu

Đây là nội dung trọng tâm của đồ án. Mục này chứng minh tính đúng đắn của thiết kế CSDL ở cả hai tầng: tầng Logic (phụ thuộc hàm, candidate key, chuẩn hóa, phân rã bảo toàn nối) và tầng Vật lý (schema SQL Server thật, cơ chế toàn vẹn dữ liệu, chiến lược hiệu năng và vận hành).

4.2.1 Danh mục thực thể (Entity Catalog)

Toàn bộ 20 thực thể được thiết kế từ ERD (Chương 3, mục 3.2.1.5) được triển khai thành bảng vật lý trong SQL Server. Mọi thực thể (trừ Tenant) đều có khóa ngoại TenantId để đảm bảo cô lập dữ liệu đa khách thuê.

Thực thể	Mô tả	Thuộc tính chính (rút gọn)
Tenant	Đại diện một cửa hàng/doanh nghiệp thuê hệ thống	TenantId (PK), Name, Domain, Status
Branch	Chi nhánh/kho hàng thuộc một Tenant	BranchId (PK), TenantId (FK), Name, Address
User	Tài khoản người dùng thuộc một Tenant	UserId (PK), TenantId (FK), Username, Email, Role
Partner	Nhà cung cấp/khách hàng	PartnerId (PK), TenantId (FK), Type, Name
SalesChannel	Kênh bán (POS/Shopee/TikTok/Lazada)	ChannelId (PK), TenantId (FK), Code, Type
ProductCategory	Danh mục sản phẩm, phân cấp cha-con	CategoryId (PK), TenantId (FK), ParentId
Brand	Thương hiệu sản phẩm	BrandId (PK), TenantId (FK), Name
Product	Sản phẩm gốc, có thể có nhiều biến thể	ProductId (PK), TenantId, CategoryId, BrandId
SKUVariant	Biến thể cụ thể, đơn vị quản lý tồn kho	SKUId (PK), TenantId, SKUCode (unique/Tenant), Price
Barcode	Mã vạch gắn với một SKU	BarcodeId (PK), TenantId, SKUId (FK), Code (unique)
StockBalance (Ledger read model)	Số dư tồn kho hiện tại theo SKU/Branch	(BranchId, SKUId) PK phức hợp, OnHand, Reserved, AvgCost
InventoryTransaction (Ledger)	Số cái tồn kho append-only	TransactionId (PK), TenantId, Type, Quantity, BalanceAfter

Thực thể	Mô tả	Thuộc tính chính (rút gọn)
Stocktake	Phiên kiểm kho	StocktakeId (PK), TenantId, BranchId, Status
StocktakeItem	Chi tiết số đếm thực tế theo SKU	StocktakeItemId (PK), CountedQty, SystemQty, VarianceQty (computed)
Order	Đơn hàng hợp nhất từ mọi kênh	OrderId (PK), TenantId, ChannelId (FK), Status
OrderItem	Chi tiết dòng sản phẩm, snapshot giá vốn	OrderItemId (PK), SKUId, CostPrice, Subtotal (computed)
PurchaseReceipt	Phiếu nhập hàng	ReceiptId (PK), TenantId, BranchId, SupplierId, Status
PurchaseReceiptItem	Chi tiết dòng hàng trong phiếu nhập	ReceiptItemId (PK), SKUId, Quantity, UnitCost
StockTransfer	Phiếu chuyển kho giữa 2 chi nhánh	TransferId (PK), FromBranchId, ToBranchId, Status
StockTransferItem	Chi tiết dòng SKU chuyển kho	TransferItemId (PK), TransferId, SKUId, Quantity

Bảng 4.1: Danh mục 20 thực thể triển khai trong schema SQL Server

4.2.2 Chứng minh Thiết kế Logic (Logical Design Correctness)

Trước khi trình bày DDL, mục này chứng minh tính đúng đắn của mô hình dữ liệu ở tầng logic — độc lập với SQL Server — theo đúng 5 tiêu chí: phụ thuộc hàm, candidate key, chuẩn hóa (3NF/BCNF), phân rã bảo toàn nối, và không tạo dữ liệu dư thừa không cần thiết.

4.2.2.1 Phụ thuộc hàm và Candidate Key

Vì mô hình được xây dựng theo phương pháp ánh xạ Thực thể → Quan hệ (mỗi thực thể là một bảng riêng), tập phụ thuộc hàm (FD) của phần lớn quan hệ có dạng {PK} → {tất cả thuộc tính còn lại}, cộng thêm FD phát sinh từ các ràng buộc UNIQUE tường minh (đóng vai trò candidate key phụ). Ba ví dụ tiêu biểu có nhiều candidate key:

- **Users:** {UserId} → tất cả; {TenantId, Username} → tất cả; {TenantId, Email} → tất cả — 3 candidate key.
- **SKUVariant:** {SKUId}; {TenantId, SKUCode}; {ProductId, Color, Size} — 3 candidate key.
- **StockBalance:** chỉ có đúng 1 candidate key là khóa tự nhiên phức hợp {BranchId, SKUId} — không dùng surrogate key vì cặp này vốn đã duy nhất về nghiệp vụ.

Bảng tổng hợp candidate key cho toàn bộ 20 quan hệ được trình bày đầy đủ trong tài liệu LOGICAL_DESIGN.md đính kèm.

4.2.2.2 Chuẩn hóa: 3NF và BCNF

Định lý áp dụng (BCNF): quan hệ R đạt BCNF nếu mọi FD không tầm thường $X \rightarrow Y$ đều có X là siêu khóa. Vì mọi bảng chỉ có 2 dạng FD không tầm thường — từ PK và từ các candidate key phụ do UNIQUE sinh ra — và cả hai dạng đều có về trái là candidate key, nên toàn bộ schema đạt BCNF, ngoại trừ 2 ngoại lệ được khai báo tường minh dưới đây (không che giấu).

Ngoại lệ 1 — TenantId lặp lại trên bảng con. Về hình thức, TenantId trên StockBalance phụ thuộc vào BranchId (qua Branch.TenantId) — chỉ là *một phần* của khóa chính (*BranchId, SKUId*), tức là **phụ thuộc bộ phận**, vi phạm 2NF nếu xét tuyệt đối. Tương tự cho 9 bảng con khác. Lý do giữ lại: (1) cho phép lọc theo Tenant trực tiếp trên mọi bảng mà không cần JOIN ngược, phục vụ EF Core Global Query Filter và composite index; (2) phòng thủ theo chiều sâu — nếu một truy vấn quên điều kiện JOIN, TenantId trên chính bảng con vẫn chặn được rò rỉ dữ liệu chéo Tenant. Đây là mẫu thiết kế tiêu chuẩn của kiến trúc SaaS đa khách thuê, đánh đổi có ý thức chứ không phải sai sót — được kiểm soát bằng nguyên tắc TenantId con **không bao giờ được UPDATE độc lập** sau khi INSERT.

Ngoại lệ 2 — StockBalance là dữ liệu dẫn xuất từ InventoryTransaction. Về lý thuyết, StockBalance.OnHand suy ra được 100% từ SUM() trên InventoryTransaction. Nhóm giữ bảng này như một **read model** tách biệt (tư duy CQRS) vì: (1) truy vấn tồn khả dụng xảy ra với tần suất rất cao, không thể SUM() toàn bộ lịch sử mỗi lần đọc; (2) StockBalance là mục tiêu **khóa hàng** bắt buộc trong cơ chế chống bán vượt tồn (mục 4.2.6) — cần một bảng đúng 1 dòng/SKU/Branch để áp dụng lock hint. Rủi ro lệch dữ liệu được kiểm soát bằng script đối chiếu định kỳ (mục 4.3.3).

4.2.2.3 Phân rã Bảo toàn Nối (Lossless-Join Decomposition)

Điều kiện đủ (Chandra–Merlin): phân rã R thành R1, R2 là bảo toàn nối nếu $R1 \cap R2$ là khóa của R1 hoặc R2.

Vì **mọi khóa ngoại trong schema đều trở đến khóa chính** của bảng cha (không có FK nào trở tới cột không phải khóa), điều kiện Chandra–Merlin được thỏa mãn tự động cho toàn bộ **44 cặp bảng cha-con** (45 cột FK) trong schema — chứng minh chi tiết từng cặp được trình bày trong LOGICAL_DESIGN.md mục 6. Bảo toàn phụ thuộc (dependency-preserving) cũng đạt, vì mọi FD gốc của một thực thể nằm trọn trong đúng 1 bảng, ngoại trừ 2 ngoại lệ đã khai báo ở trên.

4.2.2.4 Loại bỏ dữ liệu dư thừa không cần thiết

Khác với 2 ngoại lệ có chủ đích ở trên, các cột sau **không có lý do chính đáng** để lưu trùng nên được thiết kế dưới dạng **computed column** (SQL Server tự tính, không lưu vật lý dư thừa, không có nguy cơ dị thường cập nhật):

- `OrderItem.Subtotal = Quantity × UnitPrice`
- `StocktakeItem.VarianceQty = CountedQty − SystemQty`
- `StockBalance.Available = OnHand − Reserved` (đồng thời phục vụ đánh index cho cảnh báo tồn thấp)

Riêng `Orders.TotalAmount` (cache của `SUM(OrderItem.Subtotal)`) được giữ lại làm cột lưu trùng **có chủ đích** (không phải computed column) vì lý do hiệu năng hiển thị danh sách đơn hàng — cùng nhóm ngoại lệ với StockBalance ở trên, được kiểm soát bằng script đối chiếu (mục 4.3.2).

4.2.3 Script DDL (Định nghĩa bảng)

Dưới đây là các bảng tiêu biểu nhất, trích từ file `schema.sql` đầy đủ (20 bảng, phiên bản 2.0). Quy ước chung: khóa chính dùng UNIQUEIDENTIFIER (qua NEWID()) thay vì số tự tăng; mọi cột văn bản dùng NVARCHAR để đảm bảo lưu đúng dấu tiếng Việt.

```
CREATE TABLE tenants (
    tenant_id    UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
    name        NVARCHAR(255) NOT NULL,
```

```

    domain      VARCHAR(100)   NULL,
    status       VARCHAR(20)    NOT NULL DEFAULT 'ACTIVE'
                CHECK (status IN ('ACTIVE','SUSPENDED','INACTIVE')),
    created_at   DATETIME       NOT NULL DEFAULT GETDATE(),
    updated_at   DATETIME       NOT NULL DEFAULT GETDATE(),
    CONSTRAINT uq_tenants_domain UNIQUE (domain)
);

-- Kênh bán (Omnichannel): hình thức hoa thành thực thể tra cứu thay vì
-- 1 cột text tu do trên orders, phục vụ trực tiếp FR-REP-01
CREATE TABLE sales_channels (
    channel_id   UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
    tenant_id    UNIQUEIDENTIFIER NOT NULL FOREIGN KEY REFERENCES tenants(
        tenant_id),
    code         VARCHAR(30)    NOT NULL,
    name         NVARCHAR(100)  NOT NULL,
    type         VARCHAR(20)    NOT NULL CHECK (type IN ('OFFLINE','ONLINE')),
    is_active    BIT NOT NULL DEFAULT 1,
    created_at   DATETIME NOT NULL DEFAULT GETDATE(),
    CONSTRAINT uq_channels_tenant_code UNIQUE (tenant_id, code)
);

CREATE TABLE users (
    user_id      UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
    tenant_id    UNIQUEIDENTIFIER NOT NULL FOREIGN KEY REFERENCES tenants(
        tenant_id),
    username     VARCHAR(100)   NOT NULL,
    email        VARCHAR(255)   NOT NULL,
    password_hash VARCHAR(255)  NOT NULL,
    role         VARCHAR(50)    NOT NULL CHECK (role IN ('Owner','Staff','
        Cashier')),
    created_at   DATETIME NOT NULL DEFAULT GETDATE(),
    CONSTRAINT uq_users_tenant_username UNIQUE (tenant_id, username),
    CONSTRAINT uq_users_tenant_email    UNIQUE (tenant_id, email)
);

```

```

CREATE TABLE sku_variants (
    sku_id       UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
    tenant_id    UNIQUEIDENTIFIER NOT NULL FOREIGN KEY REFERENCES tenants(
        tenant_id),
    product_id   UNIQUEIDENTIFIER NOT NULL FOREIGN KEY REFERENCES products(
        product_id),
    sku_code     VARCHAR(50)    NOT NULL,
    price        DECIMAL(15,2) NOT NULL CHECK (price >= 0),
    created_at   DATETIME NOT NULL DEFAULT GETDATE(),
    CONSTRAINT uq_sku_tenant_code UNIQUE (tenant_id, sku_code),
    CONSTRAINT uq_sku_variant_combo UNIQUE (product_id, color, size)
);

```

```

-- "Read model" tồn kho: PK là khoa tự nhiên (branch_id, sku_id) -- xem
-- mục 4.2.2 (chứng minh Logic) và mục 4.2.6 (Concurrency Control)

```

```

CREATE TABLE stock_balances (
    tenant_id    UNIQUEIDENTIFIER NOT NULL FOREIGN KEY REFERENCES tenants(

```

```

        tenant_id),
branch_id    UNIQUEIDENTIFIER NOT NULL FOREIGN KEY REFERENCES branches(
    branch_id),
sku_id       UNIQUEIDENTIFIER NOT NULL FOREIGN KEY REFERENCES sku_variants(
    sku_id),
on_hand      NUMERIC(14,3) NOT NULL DEFAULT 0,
reserved     NUMERIC(14,3) NOT NULL DEFAULT 0,
avg_cost     NUMERIC(14,4) NOT NULL DEFAULT 0,
available    AS (on_hand - reserved) PERSISTED,
updated_at   DATETIME NOT NULL DEFAULT GETDATE(),
CONSTRAINT pk_stock_balances PRIMARY KEY (branch_id, sku_id),
CONSTRAINT chk_stock_available_non_negative CHECK (on_hand - reserved >= 0)
);

```

```

-- So cai append-only -- nguồn su that duy nhat ve ton kho
CREATE TABLE inventory_transactions (
    transaction_id    UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
    tenant_id         UNIQUEIDENTIFIER NOT NULL FOREIGN KEY REFERENCES tenants
        (tenant_id),
    branch_id         UNIQUEIDENTIFIER NOT NULL FOREIGN KEY REFERENCES
        branches(branch_id),
    sku_id            UNIQUEIDENTIFIER NOT NULL FOREIGN KEY REFERENCES
        sku_variants(sku_id),
    transaction_type   VARCHAR(3) NOT NULL CHECK (transaction_type IN ('IN', '
        OUT')),
    quantity           NUMERIC(14,3) NOT NULL CHECK (quantity > 0),
    balance_after      NUMERIC(14,3) NOT NULL,
    reference_id       UNIQUEIDENTIFIER NULL,
    reference_type     VARCHAR(30) NULL CHECK (reference_type IS NULL OR
        reference_type IN
            ('ORDER', 'PURCHASE_RECEIPT', 'STOCK_TRANSFER', '
            STOCKTAKE_ADJUSTMENT')),
    created_at         DATETIME NOT NULL DEFAULT GETDATE()
    -- KHONG co updated_at / deleted_at: bang append-only, xem muc 4.2.4
);

```

```

-- Kiem kho (FR-INV-04) -- bo sung sau khi phat hien bi bo sot hoan toan
CREATE TABLE stocktakes (
    stocktake_id      UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
    tenant_id         UNIQUEIDENTIFIER NOT NULL FOREIGN KEY REFERENCES tenants(
        tenant_id),
    branch_id         UNIQUEIDENTIFIER NOT NULL FOREIGN KEY REFERENCES branches(
        branch_id),
    status            VARCHAR(20) NOT NULL DEFAULT 'InProgress'
        CHECK (status IN ('InProgress', 'Completed')),
    created_at        DATETIME NOT NULL DEFAULT GETDATE()
);

```

```

-- Chan 2 phien kiem kho InProgress dong thoi tren cung 1 chi nhanh
CREATE UNIQUE NONCLUSTERED INDEX uq_stocktake_one_active_per_branch
    ON stocktakes (branch_id) WHERE status = 'InProgress';

```

```

CREATE TABLE orders (
    order_id          UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
    tenant_id         UNIQUEIDENTIFIER NOT NULL FOREIGN KEY REFERENCES tenants(
        tenant_id),
    branch_id         UNIQUEIDENTIFIER NOT NULL FOREIGN KEY REFERENCES branches(
        branch_id),
    channel_id        UNIQUEIDENTIFIER NOT NULL FOREIGN KEY REFERENCES
        sales_channels(channel_id),
    status            VARCHAR(20) NOT NULL DEFAULT 'Draft'
                    CHECK (status IN ('Draft', 'Reserved', 'Confirmed', 'Completed',
                    'Cancelled')),
    total_amount      DECIMAL(15,2) NOT NULL DEFAULT 0 CHECK (total_amount >= 0),
    created_at        DATETIME NOT NULL DEFAULT GETDATE()
);

CREATE TABLE order_items (
    order_item_id     UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
    tenant_id         UNIQUEIDENTIFIER NOT NULL FOREIGN KEY REFERENCES tenants(
        tenant_id),
    order_id          UNIQUEIDENTIFIER NOT NULL FOREIGN KEY REFERENCES orders(
        order_id),
    sku_id            UNIQUEIDENTIFIER NOT NULL FOREIGN KEY REFERENCES
        sku_variants(sku_id),
    quantity          NUMERIC(14,3) NOT NULL CHECK (quantity > 0),
    unit_price         DECIMAL(15,2) NOT NULL CHECK (unit_price >= 0),
    cost_price         DECIMAL(15,4) NOT NULL CHECK (cost_price >= 0),
    subtotal          AS (quantity * unit_price) PERSISTED,
    CONSTRAINT uq_order_items_order_sku UNIQUE (order_id, sku_id)
);

```

14 bảng còn lại (Branch, Partner, ProductCategory, Brand, Product, Barcode, StocktakeItem, PurchaseReceipt, PurchaseReceiptItem, StockTransfer, StockTransferItem...) theo đúng cấu trúc tương tự, xem đầy đủ trong file *schema.sql* đính kèm.

4.2.4 Cơ chế Append-only cho Sổ cái Tồn kho

Hiện thực hóa BR-01 và NFR-SEC-03: bảng *inventory_transactions* tuyệt đối không cho phép UPDATE/DELETE, kể cả thao tác SQL trực tiếp. SQL Server không hỗ trợ BEFORE TRIGGER — cơ chế chặn dùng INSTEAD OF TRIGGER, kèm SET XACT_ABORT ON để đảm bảo rollback nhất quán toàn batch:

```

CREATE TRIGGER trg_block_ledger_modification
ON inventory_transactions
INSTEAD OF UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;
    RAISERROR (
        N'inventory_transactions is append-only. % is not allowed. '
        'Use a compensating entry instead.', 16, 1);
    ROLLBACK TRANSACTION;
END;

```

Mọi điều chỉnh sai sót phải thực hiện bằng dòng bù trừ mới (compensating entry) — đúng tinh thần ”chỉ ghi thêm, không sửa” của sổ cái kế toán. Bảng chứng kiểm thử tại mục 4.3.2.

4.2.5 Cô lập Đa khách thuê — Tenant Isolation

Hệ thống áp dụng 2 lớp phòng thủ:

Lớp 1 (đã có trong schema): cột `tenant_id` trên mọi bảng nghiệp vụ (kể cả 9 bảng con vốn có thể suy ra qua FK — xem ngoại lệ chuẩn hóa mục 4.2.2), làm cơ sở cho EF Core Global Query Filter khi lập trình Backend.

Lớp 2 (đã thiết kế, chờ Backend kích hoạt): SQL Server Row-Level Security (RLS) — ép buộc cô lập Tenant ngay tại tầng CSDL, độc lập với việc Backend có viết đúng `WHERE tenant_id = ...` hay không:

```
CREATE FUNCTION dbo.fn_tenant_predicate(@tenant_id UNIQUEIDENTIFIER)
RETURNS TABLE WITH SCHEMABINDING AS
RETURN SELECT 1 AS result
WHERE @tenant_id = CAST(SESSION_CONTEXT(N'TenantId') AS UNIQUEIDENTIFIER);

CREATE SECURITY POLICY TenantIsolationPolicy
ADD FILTER PREDICATE dbo.fn_tenant_predicate(tenant_id) ON dbo.orders,
ADD FILTER PREDICATE dbo.fn_tenant_predicate(tenant_id) ON dbo.
    inventory_transactions
WITH (STATE = ON);
```

Backend đặt `SESSION_CONTEXT('TenantId', @tenant_id)` ngay sau khi xác thực JWT. Chưa kích hoạt trong phạm vi đồ án vì cần Backend thật để set giá trị này.

4.2.6 Kiểm soát đồng thời — Chống bán vượt tồn (Inventory Reservation)

Giải quyết trực tiếp NFR-PERF-02. SQL Server mặc định `READ COMMITTED`, không đủ chống race-condition. Nhóm áp dụng **Pessimistic Locking** bằng `WITH (UPDLOCK, ROWLOCK)` trên đúng PK tự nhiên của `stock_balances`:

```
BEGIN TRANSACTION;

SELECT on_hand, reserved
FROM stock_balances WITH (UPDLOCK, ROWLOCK)
WHERE branch_id = @branch_id AND sku_id = @sku_id;

-- Kiểm tra (on_hand - reserved) >= @requested_qty o tang ung dung
-- Neu khong du -> ROLLBACK, tra loi 409 Conflict

UPDATE stock_balances
SET reserved = reserved + @requested_qty, updated_at = GETDATE()
WHERE branch_id = @branch_id AND sku_id = @sku_id;

COMMIT TRANSACTION;
```

Vì PK là khóa tự nhiên (`branch_id, sku_id`), câu lệnh khóa hàng seek trực tiếp trên Clustered Index — không cần thêm 1 bước lookup như khi dùng surrogate key. Với 50 giao dịch đồng thời tranh 1 SKU còn 1 tồn, SQL Server tuần tự hóa: chỉ giao dịch đầu tiên thành công, các giao dịch sau thấy `reserved` đã cập nhật nên tự động bị từ chối — không có khả năng lost update. `CHECK (on_hand - reserved >= 0)` là lưới an toàn cuối cùng ở tầng CSDL.

4.2.7 Giá vốn bình quân (WAC) và Snapshot giá vốn đơn hàng

`stock_balances.avg_cost` được cập nhật theo công thức bình quân gia quyền mỗi khi `purchase_receipts` chuyển Confirmed, trong cùng transaction với việc ghi Ledger (đảm bảo atomic — NFR-SEC-02):

$$\text{New Cost} = \frac{(\text{Old Stock} \times \text{Old Cost}) + (\text{Import Qty} \times \text{Import Price})}{\text{Old Stock} + \text{Import Qty}}$$

`order_items.cost_price` snapshot `avg_cost` tại thời điểm đơn Confirmed — NOT NULL, bất biến sau khi ghi (BR-04), đảm bảo báo cáo lợi nhuận không bị thay đổi ngược thời gian khi giá vốn trung bình biến động sau đó.

4.2.8 Chỉ mục (Index) theo Workload

Nguyên tắc: phân tích mẫu truy vấn thực tế thay vì đánh index tràn lan (mỗi index thừa làm chậm INSERT trên bảng ghi nhiều như `inventory_transactions`).

```
CREATE NONCLUSTERED INDEX idx_ledger_tenant_created
  ON inventory_transactions (tenant_id, created_at);
CREATE NONCLUSTERED INDEX idx_ledger_tenant_sku
  ON inventory_transactions (tenant_id, sku_id);
CREATE NONCLUSTERED INDEX idx_orders_tenant_status
  ON orders (tenant_id, status) INCLUDE (created_at);
CREATE NONCLUSTERED INDEX idx_stock_tenant_available
  ON stock_balances (tenant_id, available);
```

Lý do chọn thứ tự cột: `TenantId` luôn đứng đầu vì mọi truy vấn multi-tenant đều lọc theo Tenant trước tiên — giúp SQL Server thu hẹp phạm vi quét ngay từ prefix của composite index. Bảng `purchase_receipt_items`, `stock_transfer_items`, `stocktake_items` **không** được đánh thêm index ngoài UNIQUE bắt buộc, vì là bảng chi tiết khối lượng ghi thấp, luôn truy vấn qua bảng cha trước.

4.2.9 Chiến lược Backup / Recovery

Bảng 4.2: Chiến lược backup/recovery đề xuất

Hạng mục	Khuyến nghị
Recovery model	FULL — bắt buộc vì Ledger tài chính không được mất giao dịch, cần point-in-time recovery
Full backup	Hàng tuần (hàng ngày ở giai đoạn phát triển)
Differential backup	Hàng ngày
Transaction log backup	Mỗi 15 phút — bảng <code>orders/inventory_transactions</code> ghi liên tục
RPO / RTO	RPO ≤ 15 phút (khớp chu kỳ log backup); RTO ≤ 1 giờ (môi trường đồ án)
Retention	Tối thiểu 30 ngày (full/differential), 7 ngày (log)
Kiểm thử phục hồi	Diễn tập restore định kỳ (hàng quý) lên môi trường tách biệt

Đây là khuyến nghị thiết kế — việc thực thi lịch backup thật cần môi trường production, ngoài phạm vi bàn giao học kỳ này (mục 1.6).

4.2.10 Đánh giá OLTP (Online Transaction Processing)

Workload của OISM là giao dịch ngắn, tần suất cao (Reserve/Confirm/Deduct), mỗi giao dịch chỉ đụng 1-3 bảng — schema chuẩn hóa 3NF/BCNF (mục 4.2.2) là lựa chọn đúng cho OLTP, ưu tiên nhất quán ghi hơn tốc độ đọc tổng hợp, ngược lại hoàn toàn với mô hình OLAP/star-schema phi chuẩn hóa để tối ưu đọc.

Điểm nóng tranh chấp: `stock_balances` là bảng nhiều giao dịch cùng chờ khóa nhất (Flash Sale, NFR-PERF-02) — đã xử lý bằng `WITH (UPDLOCK, ROWLOCK)` mức dòng (mục 4.2.6).

Đọc báo cáo chặn ghi OLTP: mặc định `READ COMMITTED` có thể khiến truy vấn báo cáo dài chặn giao dịch ghi và ngược lại. Khuyến nghị bật `READ_COMMITTED_SNAPSHOT` để Reporting dùng row-versioning thay vì xin khóa:

```
ALTER DATABASE OISM SET READ_COMMITTED_SNAPSHOT ON;
```

Đánh đổi: tăng nhẹ dung lượng tempdb — chấp nhận được để giảm blocking giữa OLTP và Reporting. Khi hệ thống có nhiều Tenant lớn đồng thời, nên tách Reporting sang read-replica/data warehouse riêng — ngoài phạm vi đồ án, ghi nhận là hướng mở rộng.

4.2.11 Đánh giá Partitioning (đánh giá, chưa triển khai)

Bảng có khả năng cần partition trong tương lai: `inventory_transactions` — bảng duy nhất tăng trưởng không giới hạn (append-only, không bao giờ xóa). Chiến lược đề xuất khi cần: RANGE theo tháng trên `created_at`:

```
CREATE PARTITION FUNCTION pf_ledger_monthly (DATETIME)
AS RANGE RIGHT FOR VALUES ('2026-01-01', '2026-02-01', ...);
CREATE PARTITION SCHEME ps_ledger_monthly
AS PARTITION pf_ledger_monthly ALL TO ([PRIMARY]);
```

Tiêu chí quyết định triển khai thật (không làm ngay bây giờ): số dòng vượt ~10-20 triệu, HOẶC độ trễ báo cáo (p95) vượt ngưỡng NFR-PERF-01 đo được thực tế, HOẶC 1 Tenant chiếm tỷ trọng dữ liệu bất thường lớn. Ở quy mô đồ án/giai đoạn khởi điểm SaaS, 2 index composite (`tenant_id, created_at`) và (`tenant_id, sku_id`) đã đủ đáp ứng — triển khai partitioning sớm khi chưa cần là over-engineering, đúng tinh thần ”đánh giá nhưng chỉ triển khai khi quy mô cần”.

4.2.12 Dữ liệu mẫu (Seed Data)

Bộ dữ liệu mẫu gồm 2–3 Tenant độc lập (kiểm chứng NFR-TENANT-01), mỗi Tenant có User theo cả 3 vai trò, sản phẩm/SKU với tồn kho ban đầu, kịch bản Purchase Receipt 2 đợt giá khác nhau (kiểm chứng WAC), đơn hàng mẫu ở đủ trạng thái Draft/Reserved/Confirmed/Completed/Cancelled qua nhiều kênh bán (kiểm chứng tính Omnichannel), và kịch bản Stocktake/StockTransfer minh họa 2 bảng mới bổ sung.

4.3 Kiểm thử Cơ sở dữ liệu

4.3.1 Phạm vi kiểm thử

Vì hệ thống chưa có Backend thật (mục 1.6), kiểm thử trong phạm vi đồ án diễn ra hoàn toàn ở **mức cơ sở dữ liệu** — chạy trực tiếp script T-SQL trên SQL Server Management Studio (SSMS) để kiểm chứng các ràng buộc và công thức nghiệp vụ trọng yếu:

- Ràng buộc append-only trên `inventory_transactions` (thử UPDATE/DELETE, kỳ vọng bị chặn bởi INSTEAD OF TRIGGER).

- Công thức giá vốn bình quân gia quyền (so sánh giá trị avg_cost tính toán với giá trị kỳ vọng theo kịch bản nhập hàng nhiều đợt ở mục 4.2.7).
- Ràng buộc available = on_hand - reserved không âm (đã được ép buộc ngay ở tầng schema bằng CHECK constraint, xem mục 4.2.3).
- Tính duy nhất của sku_code trong phạm vi Tenant (BR-03) và tính duy nhất toàn hệ thống của barcodes.code.
- Cô lập dữ liệu đa tenant: truy vấn chéo TenantId phải trả về rỗng.
- Đối chiếu tổng Ledger với StockBalance (kiểm chứng tính đúng đắn của "read model" đã phân tích ở mục 4.2.2).

Không nằm trong phạm vi kiểm thử của học kỳ này: Kiểm thử tải thực tế qua API (hiện chỉ mô phỏng bằng nhiều phiên SSMS song song như mục 4.3.7), kiểm thử giao diện (chưa có Frontend), kiểm thử tích hợp Webhook thật.

4.3.2 Kiểm chứng ràng buộc Append-only

Chạy trực tiếp 2 câu lệnh dưới đây trên một dòng inventory_transactions đã tồn tại từ seed data, cả hai đều **phải thất bại** (hoặc chính xác hơn: không có tác dụng gì, vì bị INSTEAD OF TRIGGER thay thế):

```
-- Test 1: thu UPDATE trực tiếp
UPDATE inventory_transactions
SET quantity = 999
WHERE id = @existing_transaction_id;
-- Ket qua ky vong:
-- Msg 50000, Level 16, State 1
-- inventory_transactions is append-only. UPDATE/DELETE is not allowed.
-- Use a compensating entry instead.
-- (0 rows affected) -- du lieu KHONG bi thay doi

-- Test 2: thu DELETE trực tiếp
DELETE FROM inventory_transactions
WHERE id = @existing_transaction_id;
-- Ket qua ky vong: cung thong bao loi tren, (0 rows affected)
```

Cả hai câu lệnh đều bị trigger trg_block_ledger_modification (mục 4.2.4) chặn lại — xác nhận NFR-SEC-03 được thực thi đúng ở tầng cơ sở dữ liệu, không phụ thuộc vào việc Backend có kiểm tra hay không.

4.3.3 Kiểm chứng cô lập dữ liệu đa Tenant

Truy vấn đối chiếu tenant_id giữa bảng cha-con — kỳ vọng **0 dòng** trả về (không có bản ghi nào bị lệch Tenant do lỗi ứng dụng):

```
SELECT o.order_id, o.tenant_id AS order_tenant, b.tenant_id AS branch_tenant
FROM orders o JOIN branches b ON o.branch_id = b.branch_id
WHERE o.tenant_id <> b.tenant_id;
-- Ket qua ky vong: 0 dong

SELECT sb.branch_id, sb.sku_id, sb.tenant_id AS balance_tenant, sv.tenant_id AS
    sku_tenant
FROM stock_balances sb JOIN sku_variants sv ON sb.sku_id = sv.sku_id
```

```
WHERE sb.tenant_id <> sv.tenant_id;
-- Ket qua ky vong: 0 dong
```

Lập lại truy vấn tương tự cho toàn bộ 9 bảng con có tenant_id denormalized (mục 4.2.2, Ngoại lệ 1) để đảm bảo tính nhất quán được duy trì trong suốt vòng đời dữ liệu.

4.3.4 Kiểm chứng tính toàn vẹn Ledger ↔ StockBalance

Script dưới đây đối chiếu số dư trong stock_balances với tổng cộng dồn từ toàn bộ lịch sử inventory_transactions — nếu thiết kế và dữ liệu đúng, truy vấn phải trả về **0 dòng** (không có sai lệch):

```
SELECT
    sb.sku_id,
    sb.branch_id,
    sb.on_hand AS balance_on_hand,
    SUM(CASE WHEN it.transaction_type = 'IN' THEN it.quantity
            ELSE -it.quantity END) AS ledger_computed_on_hand
FROM stock_balances sb
JOIN inventory_transactions it
    ON it.sku_id = sb.sku_id AND it.branch_id = sb.branch_id
GROUP BY sb.sku_id, sb.branch_id, sb.on_hand
HAVING sb.on_hand <>
    SUM(CASE WHEN it.transaction_type = 'IN' THEN it.quantity
            ELSE -it.quantity END);
-- Ket qua ky vong: 0 dong (khong co SKU nao bi lech)
```

4.3.5 Kiểm chứng tổng tiền đơn hàng (Orders.TotalAmount)

Đối chiếu cột cache orders.total_amount (mục 4.2.2, Ngoại lệ 2) với tổng thực tế từ order_items.subtotal — kỳ vọng **0 dòng** lệch:

```
SELECT o.order_id, o.status, o.total_amount,
    SUM(oi.subtotal) AS item_total,
    o.total_amount - SUM(oi.subtotal) AS difference
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
GROUP BY o.order_id, o.status, o.total_amount
HAVING o.total_amount <> SUM(oi.subtotal);
-- Ket qua ky vong: 0 dong
```

4.3.6 Kiểm chứng công thức giá vốn bình quân

Dựa trên kịch bản seed data 2 đợt nhập tại mục 4.2.7 (100 đơn vị giá 50.000đ, sau đó thêm 50 đơn vị giá 56.000đ), giá trị avg_cost kỳ vọng được tính tay như sau:

$$\text{New Cost} = \frac{(100 \times 50.000) + (50 \times 56.000)}{100 + 50} = \frac{5.000.000 + 2.800.000}{150} = 52.000 \text{ đ}$$

Script kiểm tra:

```
SELECT sku_id, branch_id, avg_cost
FROM stock_balances
WHERE sku_id = @sku_id AND branch_id = @branch_id;
-- Ket qua ky vong: avg_cost = 52000.0000
```

4.3.7 Mô phỏng Concurrency (kịch bản Flash Sale)

Để kiểm chứng cơ chế ở mục 4.2.6 tại mức thiết kế (chưa có Backend để test tải qua API thật), nhóm mở 2 cửa sổ Query trong SSMS (mô phỏng 2 phiên kết nối song song), mỗi phiên mô phỏng một request Reserve trên cùng một SKU chỉ còn 1 đơn vị tồn:

```
-- Cua so A (chay truoc, giu khoa):
BEGIN TRANSACTION;
SELECT on_hand, reserved FROM stock_balances WITH (UPDLOCK, ROWLOCK)
WHERE sku_id = 'sku-flash-sale';
-- (chua COMMIT, giu khoa)

-- Cua so B (chay dong thoi, cung SKU):
BEGIN TRANSACTION;
SELECT on_hand, reserved FROM stock_balances WITH (UPDLOCK, ROWLOCK)
WHERE sku_id = 'sku-flash-sale';
-- Cua so B bi treo (block), phai cho toi khi Cua so A COMMIT/ROLLBACK
```

Quan sát kỳ vọng: Cửa sổ B phải ở trạng thái chờ (hiển thị "Executing query..." không trả kết quả) cho đến khi Cửa sổ A thực hiện COMMIT TRANSACTION hoặc ROLLBACK TRANSACTION — xác nhận SQL Server tuân tự hóa đúng 2 giao dịch tranh chấp trên cùng 1 dòng, không có hiện tượng 2 giao dịch cùng đọc được reserved cũ và cùng ghi đè (lost update). Có thể quan sát trực quan hơn qua view hệ thống sys.dm_tran_locks để thấy Cửa sổ B đang ở trạng thái WAIT chờ khóa UPDLOCK mà Cửa sổ A đang giữ.

4.3.8 Kết quả kiểm thử

Bảng 4.3: Tổng hợp kết quả kiểm thử mức Database

Hạng mục kiểm thử	Số test case	Pass	Fail	Ghi chú
Schema Validation (DDL chạy sạch trên container mới)	–	–	–	Điền số liệu thực tế sau khi chạy
Append-only Constraint (mục 4.3.2)	–	–	–	
Cô lập dữ liệu đa Tenant (mục 4.3.3)	–	–	–	
Toàn vẹn Ledger ↔ StockBalance (mục 4.3.4)	–	–	–	
Tổng tiền đơn hàng (mục 4.3.5)	–	–	–	
Công thức giá vốn bình quân (mục 4.3.6)	–	–	–	So sánh với 52.000đ tính tay
Ràng buộc available không âm (CHECK constraint)	–	–	–	
Concurrency simulation (mục 4.3.7)	–	–	–	Quan sát hành vi khóa qua sys.dm_tran_locks

Ghi chú: Bảng trên để trống cột số liệu — nhóm điền kết quả thật sau khi chạy xong toàn bộ script kiểm thử trên môi trường Docker SQL Server, đính kèm chi tiết đầy đủ tại file kết quả kiểm thử theo quy ước bàn giao ở Chương 2.

4.3.9 Lỗi phát hiện và hướng xử lý

Các lỗi phát hiện trong quá trình kiểm thử schema/dữ liệu được ghi nhận trên GitHub Issues theo mức độ ưu tiên (Critical/High/Medium/Low), tuân theo quy trình Quản lý chất lượng đã mô tả ở Chương 2. Lỗi liên quan đến ràng buộc append-only hoặc công thức giá vốn được xếp mức Critical vì ảnh hưởng trực tiếp đến tính đúng đắn của toàn bộ hệ thống, ưu tiên xử lý trong vòng 24 giờ.

Chương 5

Triển khai, Đánh giá và Hướng phát triển

5.1 Triển khai hệ thống

5.1.1 Phạm vi triển khai thực tế

Vì phạm vi bàn giao của học kỳ này giới hạn ở Thiết kế + Database (mục 1.6), "triển khai" trong chương này chỉ đề cập đến việc đưa **cơ sở dữ liệu** chạy được trên máy cục bộ — chưa bao gồm triển khai Backend API hay Frontend lên môi trường thật.

5.1.2 Kiến trúc triển khai Database

- **Cơ sở dữ liệu:** SQL Server 2022 đóng gói qua Docker, đảm bảo mọi thành viên và GVHD có thể khởi tạo cùng một môi trường nhất quán chỉ với một lệnh `docker compose up`.
- **Volume lưu trữ:** Named volume để dữ liệu không mất khi container được khởi động lại.
- **Script khởi tạo:** `schema.sql` → `ledger_constraints.sql` → `seed_data.sql` chạy tuần tự để có một database sẵn sàng minh họa đầy đủ nghiệp vụ.

5.1.3 Hướng dẫn cài đặt

Xem chi tiết đầy đủ tại tài liệu `installation_guide.md` đính kèm gói bàn giao cuối kỳ — bao gồm các bước: cài Docker, clone GitHub repo, chạy `docker compose up -d`, và chạy tuần tự 3 script SQL nêu trên để có database mẫu hoàn chỉnh.

5.1.4 Kiến trúc triển khai đã hoạch định cho hệ thống hoàn chỉnh

Với hệ thống hoàn chỉnh (ngoài phạm vi bàn giao hiện tại), kiến trúc triển khai đã được hoạch định trong thiết kế (Chương 3) như sau, làm cơ sở cho giai đoạn phát triển tiếp theo:

- Backend API đóng gói Docker image, triển khai qua CI/CD (GitHub Actions).
- Frontend (Admin Dashboard, POS PWA) triển khai qua CDN.
- Cơ sở dữ liệu SQL Server managed service, tách biệt môi trường development/staging/production.

5.2 Đánh giá kết quả đạt được

5.2.1 Kết quả đạt được so với kế hoạch

Trong thời hạn 5 tuần, nhóm đã hoàn thành 14 hạng mục bàn giao đã liệt kê ở Chương 2, mục 2.3:

Bảng 5.1: Đối chiếu kế hoạch và kết quả thực hiện

Hạng mục	Trạng thái	Ghi chú
SRS đầy đủ (FR, NFR, RTM)	Hoàn thành	
Architecture Design Document	Hoàn thành	
ERD, UML Class Diagram, Use Case Diagram	(điền trạng thái)	Cần chèn hình theo Phụ lục hình ảnh
Sequence Diagram (6 luồng nghiệp vụ)	(điền trạng thái)	
Đặc tả API (Swagger/Postman)	(điền trạng thái)	
Script DDL SQL Server	(điền trạng thái)	
Trigger/rule append-only cho Ledger	(điền trạng thái)	Đã kiểm chứng bằng script tự test (Chương 4)
Seed data & script kiểm tra toàn vẹn	(điền trạng thái)	
Backend API (.NET)	Chưa thực hiện	Ngoài phạm vi bàn giao học kỳ này, dời sang giai đoạn tiếp theo
Frontend (Admin Dashboard, POS PWA)	Chưa thực hiện	Ngoài phạm vi bàn giao học kỳ này

Ghi chú: Cột "Trạng thái" để trống — nhóm điền "Hoàn thành"/"Đang thực hiện" theo tiến độ thực tế trước khi nộp báo cáo cuối kỳ.

5.2.2 Đánh giá theo yêu cầu phi chức năng

Vì chưa có Backend thật, các chỉ tiêu phi chức năng về hiệu năng thời gian thực (API Response Time, khả năng chịu tải) chưa thể đo lường trực tiếp. Nhóm đã kiểm chứng được ở mức thiết kế/dữ liệu:

- Ràng buộc append-only (NFR-SEC-03): đã kiểm chứng thành công qua trigger SQL Server.
- Cô lập dữ liệu đa tenant (NFR-TENANT-01): đã kiểm chứng qua truy vấn chéo TenantId trên seed data.
- Công thức giá vốn bình quân: đã kiểm chứng đúng với kịch bản nhập hàng nhiều đợt trong seed data.
- Concurrency (NFR-PERF-02): mới mô phỏng bằng nhiều phiên SQL song song, **chưa đo lường được** ở quy mô 50 request đồng thời qua API thật — cần Backend hoàn chỉnh để kiểm thử đầy đủ.

5.2.3 Bài học kinh nghiệm

- Việc hoàn thành ERD và Data Dictionary trước khi viết DDL giúp nhóm tránh phải làm lại schema nhiều lần.
- Quyết định dùng TRIGGER thay vì RULE để chặn UPDATE/DELETE (Chương 4) giúp việc kiểm thử ràng buộc append-only rõ ràng và dễ chứng minh hơn với GVHD.

- Thu hẹp phạm vi bàn giao ngay từ đầu (chỉ Thiết kế + Database) giúp nhóm tập trung làm sâu và đúng phần lõi khó nhất của đề tài (cơ chế chống bán vượt tồn, số cái append-only), thay vì dàn trải sang Backend/Frontend mà không đủ thời gian hoàn thiện.

5.3 Hạn chế và hướng phát triển

5.3.1 Hạn chế hiện tại

Kế thừa các giới hạn đã nêu ở Chương 1 (LI-1, LI-2, LI-3), nhóm ghi nhận các hạn chế cụ thể sau khi hoàn thành thiết kế và triển khai cơ sở dữ liệu:

- **Chưa có phần mềm chạy được đầu-cuối:** Toàn bộ chức năng ở Chương 3 mới tồn tại ở dạng đặc tả và schema database, chưa thể demo trực tiếp trên giao diện người dùng.
- **Chưa kiểm thử concurrency thực tế:** Cơ chế chống bán vượt tồn (Anti-Oversell) mới được kiểm chứng ở mức mô phỏng SQL, chưa đo lường được dưới tải thật của nhiều request API đồng thời.
- **Webhook Simulator chỉ ở mức thiết kế:** Chưa có endpoint thật để nhận payload từ Shopee/TikTok Shop/Lazada.
- **Chưa có giao diện để đánh giá Usability:** Các chỉ tiêu về trải nghiệm người dùng (thời gian checkout dưới 3 click...) chưa thể đánh giá vì chưa có Frontend.

5.3.2 Hướng phát triển

- **Giai đoạn tiếp theo — Lập trình Backend:** Hiện thực Backend API (.NET 8) theo đúng Clean Architecture và schema đã thiết kế, ưu tiên hiện thực trước module Inventory Ledger và Anti-Oversell Engine vì đây là phần lõi kỹ thuật khó nhất.
- **Lập trình Frontend:** Admin Dashboard và POS Application (PWA) theo đặc tả màn hình dự kiến ở Chương 3.
- **Kiểm thử tải thực tế:** Thực hiện concurrency test với 50+ request đồng thời qua API thật để kiểm chứng đầy đủ NFR-PERF-02.
- **Tích hợp AI dự báo nhập hàng:** Như đề cương gốc đã đề xuất (Task package 5), bổ sung mô hình gợi ý số lượng đặt hàng lại dựa trên lịch sử bán hàng.
- **Tích hợp API thật của các sàn TMĐT:** Thay thế Webhook Simulator bằng kết nối thật tới Shopee Open API, TikTok Shop Partner API, Lazada Open Platform.
- **CI/CD & Containerization đầy đủ:** Thiết lập pipeline GitHub Actions để tự động build, test và deploy khi hệ thống có Backend/Frontend hoàn chỉnh.